Security Incident Detection and Response Management System

Sprint I Common Team Report

Mohannad Al Ghourayeb

Jad Moharam

Omar Tabsh

EECE 430L | 21 October 2026

GitHub repository
https://github.com/mohannad-gh/MohannadAlGhourayeb-JadMoharam-OmarTabsh

Integrated team code report and screenshots
https://github.com/mohannad-gh/MohannadAlGhourayeb-JadMoharam-OmarTabsh/tree/jad-moharam/submission

Updated team version with nine available services and verified real Users and Assets integration. All nine service implementations are present; teammate validation and evidence remain to finish. Earlier screenshots retain their original recorded context.

Table of Contents

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

Page	Section	Responsible member
3	Introduction and architecture	Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh
4	Implementation and responsibilities	Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh
5	Incident APIs	Jad Moharam
6	Notification APIs	Jad Moharam
7	Playbook APIs	Jad Moharam
8	Threat APIs	Omar Tabsh
9	Audit APIs	Omar Tabsh
10	Report APIs	Omar Tabsh
11	Users Assets Vulnerabilities integration	Mohannad Al Ghourayeb
12	Database design and relationships	Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh
13	Database entities and fields	Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh
14	Errors and authentication	Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh
15	GitHub and contributions	Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh
16	References and AI assistance	Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh
17	Native Postman screenshots	Jad Moharam
38	Database screenshots	Jad Moharam
43	Actual HTTP screenshots	Omar Tabsh
58	Current real login verification	Jad Moharam

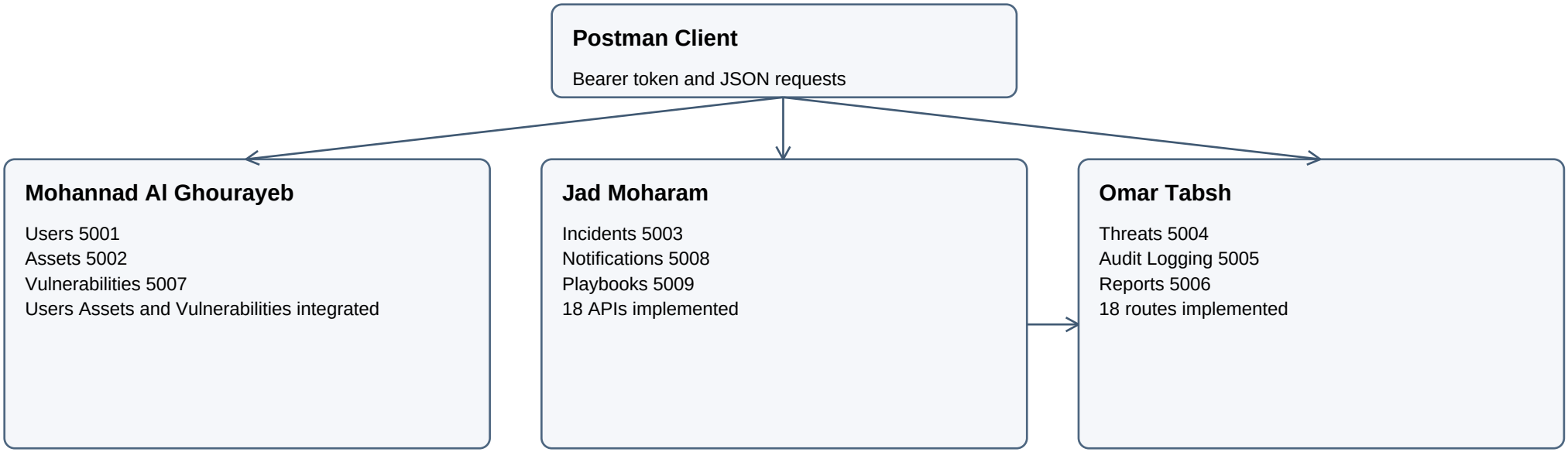
Introduction and System Architecture

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

The system centralizes incident reporting, threat information, audit records, notifications and response workflows. Its purpose is to help an organization investigate security events, assign responsibility and retain evidence of actions.

Sprint I is organized as nine Flask services using JSON over HTTP and SQLite persistence. Each team member owns three services. The integrated branch now contains nine services, including Mohannad Al Ghourayeb Users and Assets. Vulnerabilities and Asset search are now supplied in Mohannad commit 21ad9fc.

The earlier native Postman evidence uses isolated user and asset fixtures. A newer 33-request HTTP integration run verifies actual password login, real Assets and all three Jad services on one fresh SQLite database. The two evidence runs are identified separately.



JSON over HTTP between services | shared JWT verification | SQLite storage

Implementation and Team Responsibilities

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

Jad Moharam implemented the six APIs in each of Incidents, Notifications and Playbooks, their database tables, authorization and validation, durable side effects, tests, documentation and the Postman collection. Omar Tabsh implemented Threat Intelligence, Audit Logging, Security Reports and the shared authentication, configuration and audit helpers.

Mohannad Al Ghourayeb owns Users and Identity, Assets and Vulnerabilities. His Users and Assets commit 601de89 is now integrated with original authorship preserved. His follow-up 21ad9fc adds Vulnerabilities and Asset search; endpoint-level requirement review remains necessary.

The updated combined automated suite passed 220 tests; the JUnit results accompany the report. The separate native Postman run executed 47 Jad requests with 141 passing assertions, zero failures and zero errors. A later real HTTP run of Omar services produced all 20 expected statuses, including three explicit missing dependency errors.

Service and port	Owner	Current evidence
1 Users 5001	Mohannad Al Ghourayeb	Users supplied; real password login verified over HTTP
2 Assets 5002	Mohannad Al Ghourayeb	Assets supplied; creation verified over HTTP
3 Incidents 5003	Jad Moharam	6 APIs and native Postman evidence
4 Threat Intelligence 5004	Omar Tabsh	6 APIs and actual HTTP evidence
5 Audit Logging 5005	Omar Tabsh	6 APIs and actual HTTP evidence
6 Security Reports 5006	Omar Tabsh	Summary Assets Incidents and Vulnerabilities verified with real services
7 Vulnerabilities 5007	Mohannad Al Ghourayeb	6 routes supplied; real HTTP creation and report verified
8 Notifications 5008	Jad Moharam	6 APIs and native Postman evidence
9 Playbooks 5009	Jad Moharam	6 APIs and native Postman evidence

Incident Management API Documentation

Jad Moharam

Incidents store the reporter, affected asset, assigned analyst, severity and lifecycle timestamps. Users create their own reports and view their own incidents. Analysts view and update assigned incidents. Administrators assign analysts; administrators and auditors can list all incidents.

Example POST body: {"title":"Suspicious email","description":"Investigate sender","category":"Phishing","severity":"High","asset_id":1}. Assignment: {"analyst_id":2}. Note: {"content":"Headers preserved"}. Status: {"status":"Under Investigation"}.

Lifecycle: Open -> Under Investigation -> Mitigated -> Resolved -> Closed. Skips, reversals and repeats return 400. Existing users/assets are checked. Assignment requires audit availability, then queues the notification. Body values cannot override server identity or timestamps.

Method and endpoint	Inputs	Success and error behavior
POST /incidents	title, description, category, severity, asset_id; optional reporter_id	201 complete incident; 400 validation; 403 own reporter rule
GET /incidents	Admin or Auditor token	200 data array and count; 403 other roles
GET /incidents/{id}	Positive ID and permitted identity	200 incident and notes; 403 ownership; 404 absent
PUT /incidents/{id}/assign	analyst_id, active Analyst; Admin	200 assigned incident; 409 same assignment; 503 audit unavailable
PUT /incidents/{id}/status	status; Admin or assigned Analyst	200 updated incident; 400 invalid transition
POST /incidents/{id}/notes	content; Admin or assigned Analyst	201 note; 400 blank; 403 unauthorized

Notification and Alert API Documentation

Jad Moharam

Notifications preserve a user recipient, title, message, type, severity, Unread or Read status and timestamps. Analyst and User access is restricted to the recipient. Admin and authorized services can create notifications. Only Admin can delete.

Create body: {"user_id":3,"title":"Security alert","message":"Review suspicious email","type":"Security Alert","severity":"High"}. Search example: /notifications/search?user_id=3&severity=High&status=Read. Types include Incident Assignment, Security Alert, System Notification, Threat Alert and Vulnerability Alert.

The read operation needs no body and preserves the original read timestamp on repeats. Search filters combine with AND and reject unknown or repeated parameters. An existing user with no notifications receives 200 and an empty array.

Method and endpoint	Inputs and roles	Result
POST /notifications	user_id, title, message, type, severity; Admin or service key	201 notification; 400 invalid recipient or type
GET /notifications/user/{user_id}	Admin/Auditor; Analyst/User own only	200 list and count; 403 cross user
GET /notifications/{id}	Admin/Auditor; Analyst/User own only	200 notification; 403 denied; 404 absent
PUT /notifications/{id}/read	Admin or recipient Analyst/User; no body	200 Read notification and read_at
GET /notifications/search	user_id, type, severity, status; Admin/Analyst/Auditor	200 filtered array; Analyst own only; 400 bad filter
DELETE /notifications/{id}	Admin; no body	200 deleted notification_id; 403 denied; 404 absent

Playbooks and Response API Documentation

Jad Moharam

Playbooks group ordered response actions. Admin creates and deletes playbooks; Admin, Analyst and Auditor can read them. Admin and Analyst can create and progress actions. Records preserve actor identity and action history after deletion.

Playbook body: {"name":"Phishing response","description":"Investigate","incident_category":"Phishing","severity":"High"}. Action: {"description":"Preserve malicious email","step_number":1,"assigned_user_id":2}. Update: {"status":"In Progress"} then {"status":"Completed"}.

Names are case insensitive and unique. Steps are consecutive positive integers unique within a playbook. Pending -> In Progress -> Completed is enforced. Progress requires an active eligible assignee and completed preceding steps. Completed actions cannot change. Explicit completion time must be timezone aware and not in the future.

Method and endpoint	Inputs and role	Result
POST /playbooks	name, description, incident_category, severity; Admin	201 playbook; 409 duplicate name
GET /playbooks	Admin, Analyst, Auditor	200 list; 403 User
GET /playbooks/{id}	Admin, Analyst, Auditor	200 actions, history, progress; 404 absent
POST /playbooks/{id}/actions	description, step_number, optional assigned_user_id	201 action; 409 duplicate step; 400 invalid order
PUT /actions/{id}/status	status, assigned_user_id and/or completed_at	200 updated action; 400 invalid workflow
DELETE /playbooks/{id}	Admin	200 deleted playbook_id; history retained

Threat Intelligence API Documentation

Omar Tabsh

Threat Intelligence maintains IP Address, Domain, File Hash, Malware Signature and Other indicators. Values are validated by type and severity is Low, Medium, High or Critical. The unique type and value pair prevents duplicate indicators.

Create example: {"threat_type":"Domain","threat_value":"sprint-evidence.example","description":"Development evidence indicator","severity":"High","source":"Team local test"}. Update: {"severity":"Critical","description":"Confirmed development indicator"}.

Admin modifies records. Admin, Analyst and Auditor retrieve all or specific records; search follows the per-service requirement and allows Admin and Analyst. Type and value are immutable during updates. Successful changes emit audit events. The HTTP evidence uses port 5104 to isolate it from normal port 5004.

Method and endpoint	Parameters	Result
POST /threats	threat_type, threat_value, description, severity, source	201 created; 400 validation; 409 duplicate
GET /threats	No body	200 list and count
GET /threats/{id}	Positive ID	200 threat; 404 absent
GET /threats/search	type/severity exact; value/source partial	200 matching list; 400 invalid or empty filters
PUT /threats/{id}	description, severity and/or source	200 record and previous_values; last_updated set
DELETE /threats/{id}	Admin token	200 deleted record; 403 unauthorized

Security Audit Logging API Documentation

Omar Tabsh

Audit Logging stores security activities with actor, action, description, UTC timestamp, severity and originating service. Admin or trusted services create logs. Existing records cannot be updated: HTTP update methods return 405, and a SQLite trigger rejects UPDATE.

Create example: {"user_id":1,"action":"TEAM_EVIDENCE","description":"Common report evidence run","severity":"Information","related_service":"Users"}. Service calls use X-Service-Key. The key is absent from the report and screenshots.

Admin deletion creates a new LOG_DELETED record identifying the deletion. The logical users relationship does not cascade, preserving audit information when accounts change. Evidence port 5105 is separate from normal port 5005.

Method and endpoint	Roles and parameters	Result
POST /logs	Admin or service; user_id, action, description, severity, related_service	201 stored record; server assigns ID and timestamp
GET /logs	Admin, Auditor	200 newest first
GET /logs/{id}	Admin, Auditor, Analyst	200 record; 404 absent
GET /logs/search	Admin, Auditor, Analyst; user/date/action/severity/service	200 matches; 400 bad filter or date
DELETE /logs/{id}	Admin	200 deleted record and deletion audit event
GET /logs/user/{user_id}	Admin, Auditor	200 matching records or empty array

Security Reports API Documentation

Omar Tabsh

Reports compute results at request time from other services using HTTP and the caller token. Admin and Auditor can access reports. Each successful report includes generated_at and data_sources. No reports table is created.

The earlier isolated Omar run generated incident, threat and activity reports and captured dependency errors. The newer real team integration also successfully generated summary and asset reports. The latest run also verifies the actual Vulnerabilities service and its report. Historical 503 screenshots retain their original results.

Source unreachable returns 503; a bad or rejected source response returns 502. The isolated report process uses port 5106; unavailable source ports 5191, 5192 and 5197 deliberately identify the unimplemented dependencies.

GET endpoint	Sources and output	Observed status
/reports/summary	Users, Assets, Incidents, Threats; counts and open incidents	200 in newer real-login integration
/reports/incidents	Incidents; counts by severity and lifecycle status	200 actual incident data
/reports/assets	Assets; critical asset list, types and criticality	200 in newer real-login integration
/reports/threats	Threats; severity, type and five recent indicators	200 actual threat data
/reports/vulnerabilities	Vulnerabilities; severity, status and critical list	200 in newest nine-service integration
/reports/activity	Logs; activities, login events and unauthorized attempts	200 actual audit data

Users Assets and Vulnerabilities Integration

Mohannad Al Ghourayeb

Users and Assets are integrated from Mohannad commit 601de89; follow-up 21ad9fc supplies Asset search and Vulnerabilities. His original route logic is retained; Jad added package imports and an optional shared database URI. The current integration verifies bcrypt password login and actual asset creation. The earlier screenshots still use fixtures and are labeled separately.

Users must store profile data, hashed passwords, role and Active/Inactive/Suspended account status. Login must verify the password hash, reject inactive or suspended accounts, and issue the agreed JWT claims. Users and auditors have only the access allowed by the assignment.

Assets must retain identification, ownership, type and criticality and satisfy incident and vulnerability relationships. Vulnerabilities must retain CVE, CVSS, severity, affected asset and discovery source, and enforce Open -> Under Investigation -> In Progress -> Resolved -> Closed.

Before final submission, Mohannad should verify the required vulnerability lifecycle including Under Investigation, admin-only vulnerability creation, discovery/resolution fields, audit events and SQLAlchemy foreign key enforcement. Current code supplies all 18 route operations; native Postman evidence remains required. The real vulnerability report now succeeds.

A separate 30-request Flask test-client verification executed all 18 of Mohannad route operations successfully. Requirement probes confirmed that Under Investigation is rejected, Open can jump directly to Closed, and an Analyst can create vulnerabilities. These are remaining compliance issues rather than missing route files. Full sanitized results are in evidence/mohannad-api-verification.json.

Service	Required routes	Evidence status
1 Users	POST /users/register; POST /users/login; PUT /users/{id}; DELETE /users/{id}; GET /users; GET /users/{username}	6 source operations; HTTP login verified
2 Assets	POST /assets; PUT /assets/{id}; DELETE /assets/{id}; GET /assets; GET /assets/{id}; GET /assets/search	6 source operations; HTTP create and search verified
7 Vulnerabilities	POST /vulnerabilities; GET /vulnerabilities; GET /vulnerabilities/{id}; PUT /vulnerabilities/{id}; GET /vulnerabilities/search; DELETE /vulnerabilities/{id}	6 source operations; HTTP create/report verified

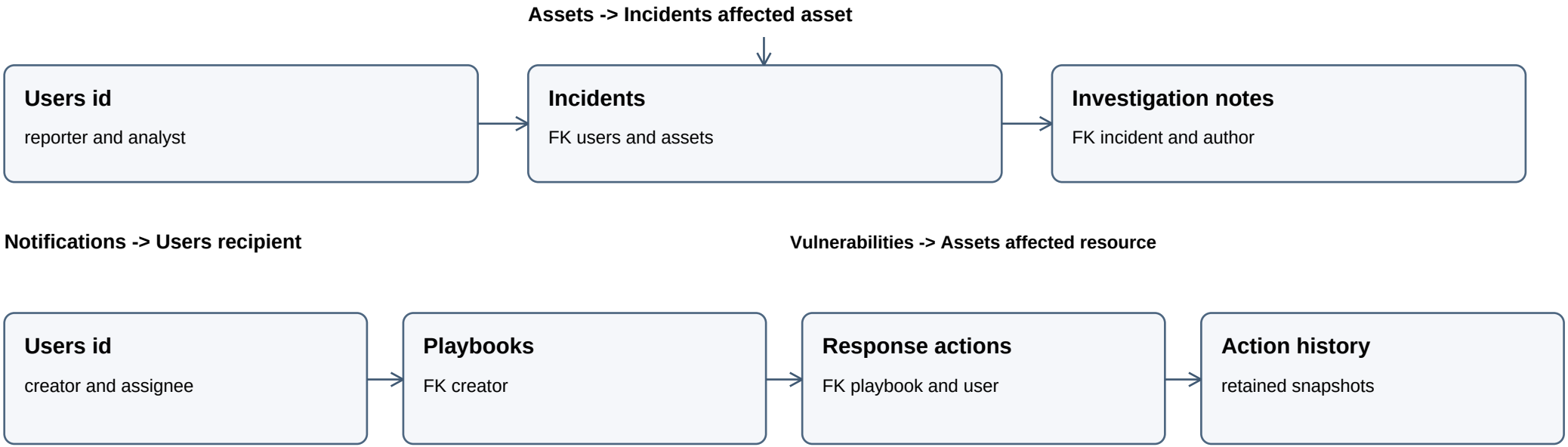
Database Design and Relationships

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

One SQLite file is the team persistence convention. Each service creates its own tables. Jad uses read only checks against users and assets for identity, account status and foreign key validation. This differs from the original API-only proposal and must be confirmed with Mohannad during final integration.

Primary keys are integer IDs. Incidents reference users as reporter and analyst and assets as the affected resource. Notes reference incidents and their author. Notifications reference recipients. Playbooks reference creators; ordered response actions reference playbooks and assignees. Action history deliberately retains snapshots after deletion.

Omar owns threats and logs. Threat type plus value is unique. Logs have an UPDATE-blocking trigger and retain a logical user link. Security Reports has no table. Historical screenshot databases use fixture schemas. The new integration database uses Mohannad real SQLAlchemy tables with id primary keys. Jad detects these and creates matching foreign keys; legacy user_id and asset_id fixtures remain supported.



Arrows show parent to related child records. History retains IDs after action deletion.

Database Entities and Fields

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

Entity and owner	Key fields	Rules
incidents Jad Moharam	incident_id PK; reporter_id FK; asset_id FK; assigned_analyst_id FK; title, description, category, severity, status, created_at, resolved_at	Constrained lifecycle and severity
investigation_notes Jad Moharam	note_id PK; incident_id FK; analyst_id FK; content, created_at	Cascade with parent incident
notifications Jad Moharam	notification_id PK; user_id FK; title, message, type, severity, status, created_at, read_at	Unread to Read; fixed type set
playbooks Jad Moharam	playbook_id PK; created_by FK; name, description, incident_category, severity, created_at	Case insensitive unique name
response_actions Jad Moharam	action_id PK; playbook_id FK; assigned_user_id FK; description, step_number, status, completed_at	Unique playbook and step; cascade parent
action_history Jad Moharam	history_id PK; action_id, playbook_id, actor_id; event, old/new status and assignee, description, step, changed_at	No cascade; evidence retained
jad_outbox Jad Moharam	event_id PK; owner_service, kind, payload, created_at, delivered_at, claimed_until	Durable pending audit/notification delivery
threats Omar Tabsh	threat_id PK; type, value, description, severity, source, date_added, last_updated	Unique type and value
logs Omar Tabsh	log_id PK; user_id logical; action, description, timestamp, severity, related_service	Immutable UPDATE trigger

Error Handling Validation and Access Control

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

Every protected request verifies a signed, unexpired JWT. Jad additionally verifies the stored active account and current role so token claims cannot preserve revoked permissions. Role and ownership checks happen before protected work. Client-supplied role fields do not grant permission.

Jad body handlers reject malformed JSON, arrays, empty bodies, unexpected fields, invalid enums, invalid integer identifiers and blank or oversized text. Values are bound as SQL parameters. Server timestamps and authenticated actor IDs are assigned by the service. Omar validates typed threat values and query filters and logs unauthorized access.

Mutation responses can report warnings for pending side effects. Assignment is cancelled if the required audit dependency is unavailable; other committed side effects use the outbox. Delivery is at least once, so retries may duplicate a remote event after an ambiguous connection failure.

The native Postman negative cases returned the expected 400, 401, 403, 404 and 409 responses. The screenshots show missing token and insufficient permissions. Newer HTTP integration evidence also verifies successful password login, wrong-password 401 and suspended-account login 403. Native Postman login screenshots remain a final team evidence task.

Role	Current permissions	Important limit
Security Administrator	Manage incidents, notifications, playbooks, threats and logs; read reports	Requires valid active identity in Jad services
Security Analyst	Assigned incidents, own notifications, playbooks/actions, threat reads/search, selected logs	No global incident listing or administrative deletion
Regular Employee or User	Own incident reports and own notifications	No playbook, threat, log or report management
Auditor	Read permitted records and reports	Cannot mutate records; follow per-service role matrix

GitHub Version Control and Contributions

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

Repository: https://github.com/mohannad-gh/MohannadAlGhourayeb-JadMoharam-OmarTabsh

Development branch: jad-moharam. Draft review: https://github.com/mohannad-gh/MohannadAlGhourayeb-JadMoharam-OmarTabsh/pull/2. Omar original commits are preserved in the merge history. The updated report, screenshots and sanitized integration log are stored under submission on the Jad branch. Original teammate merge history is preserved.

The evidence JSON was retrieved from the authenticated GitHub REST API. The corresponding HTML and screenshot are explicitly labeled as API data views. A screenshot of the signed-in GitHub website history should be added to the final package when accessible.

The table and API screenshots below retain the earlier captured history. The current Jad branch additionally merges main 3484a7c and Mohannad Users/Assets 601de89 and contains the verified integration changes. Final teammate review and main-branch merge remain outstanding.

Commit	Author	Change
f0f9959	Jad Moharam	Implement Jad Moharam services 3, 8 and 9 with all 18 APIs
f2824d8	Jad Moharam	Integrate Omar Tabsh services and shared team interfaces
db97c2f	Omar Tabsh	Document Omar Tabsh's APIs and the team integration contract
a4b329a	Omar Tabsh	Add run/dev-token scripts, database schema and Postman collection
250293d	Omar Tabsh	Implement Security Reports service (Service 6)
07f06bf	Omar Tabsh	Implement Security Audit Logging service (Service 5)
2a06e9f	Omar Tabsh	Implement Threat Intelligence service (Service 4)
bcc713a	Omar Tabsh	Add shared config, JWT authentication, response format and audit client
a620bce	Jad Moharam	Set up Flask and SQLite project with service requirements checklist
67428ca	mohannad-gh	Initial commit

References and Development Assistance

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

1. Final Project Fall 2026-2027 Sprint 1 assignment supplied by the team. Used for service allocation, endpoints, permissions, report structure and evidence requirements.

2. Earlier native Postman evidence uses commit f0f9959. The current source adds merged main 3484a7c, Mohannad 601de89 and 21ad9fc and Jad real-login integration fixes. API contracts are included in docs/JAD_API_DOCUMENTATION.md and docs/OMAR_TABSH_API_DOCUMENTATION.md. The source archive includes the current integrated code.

3. Postman collection schemas and local mode documentation: https://learning.postman.com/docs/use/use-collections/collections-schemas/ and https://learning.postman.com/latest-v-12/docs/use/native-git/setup . Used to prepare and execute the actual native collection.

4. AI assistance: Codex was used for Jad service implementation, tests, integration, documentation, actual execution automation and report preparation. Omar Git commits disclose Claude co-authorship on shared helpers, services, tests and documentation. The team must review and understand the submitted implementation and retain these disclosures.

5. Evidence provenance: native Postman requests ran on 20 October 2026 at 23:19 Beirut time. Original screenshots are framed to exclude unrelated app chrome; response pixels and results are unchanged. Actual SQLite backups follow each recorded request. The manifest has 48 entries because assignment triggered one additional notification HTTP call.

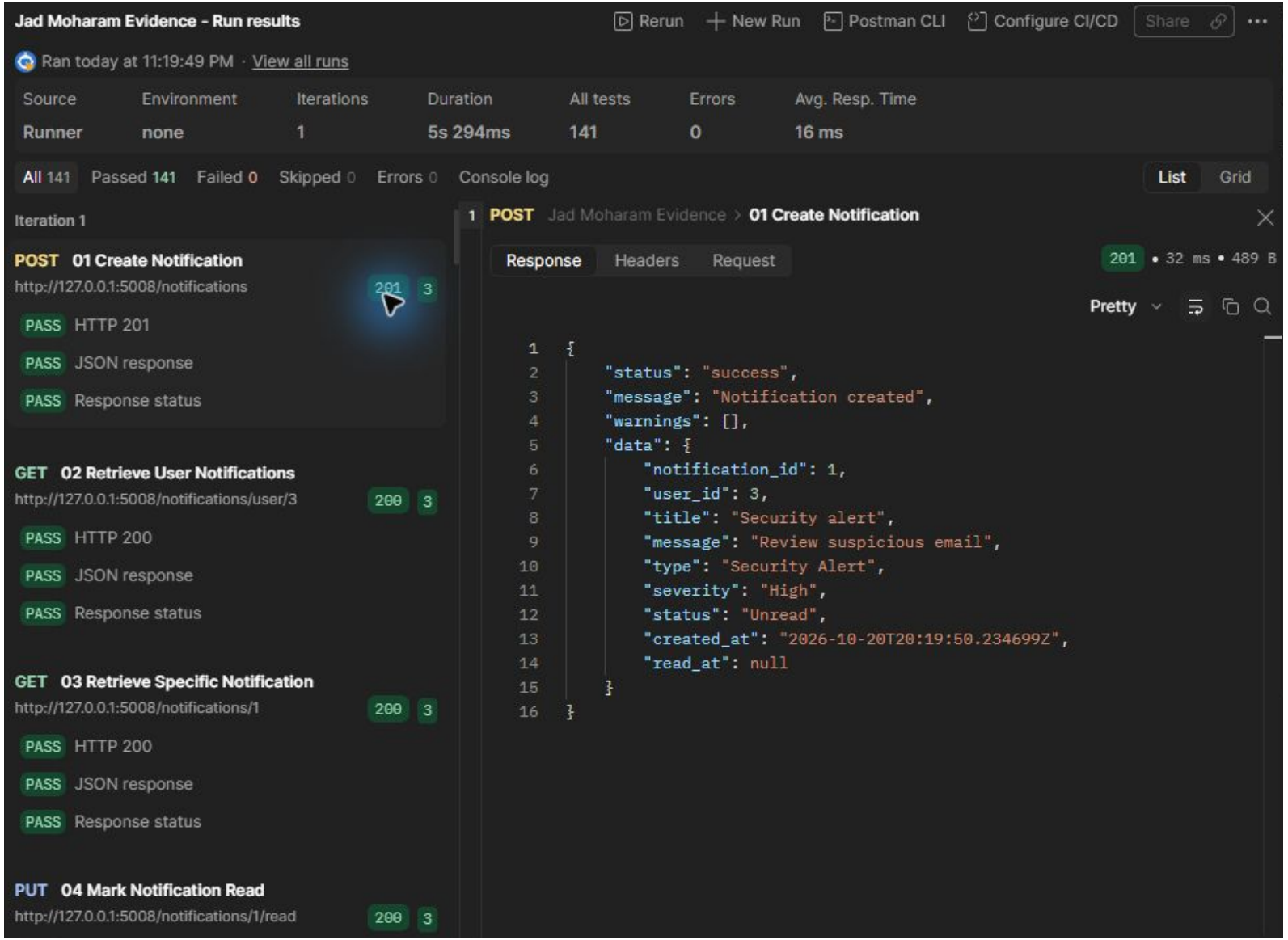
6. Omar evidence consists of actual HTTP calls viewed through a local evidence page, not native Postman screenshots. Three report successes and three dependency failures are distinguished. Remaining teammate validation and audit work, native Postman login/Omar captures and native GitHub website screenshots remain to be completed. The new real-login HTTP execution is included separately.

Create Notification

Jad Moharam

POST {{notifications_url}}/notifications

Request JSON: {"user_id":3,"title":"Security alert","message":"Review suspicious email","type":"Security Alert","severity":"High"}

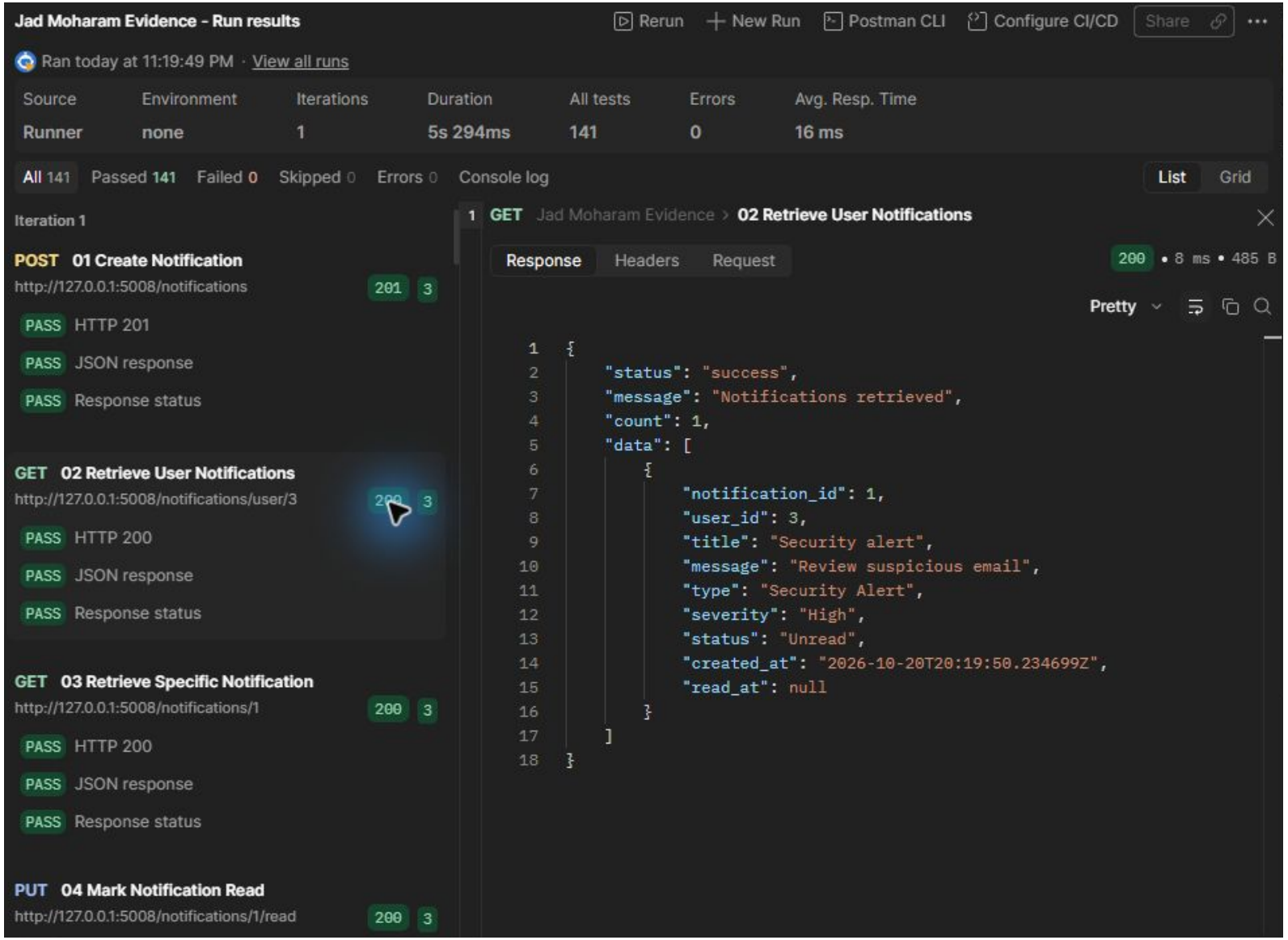


Retrieve User Notifications

Jad Moharam

GET {{notifications_url}}/notifications/user/3

Request JSON: No request body

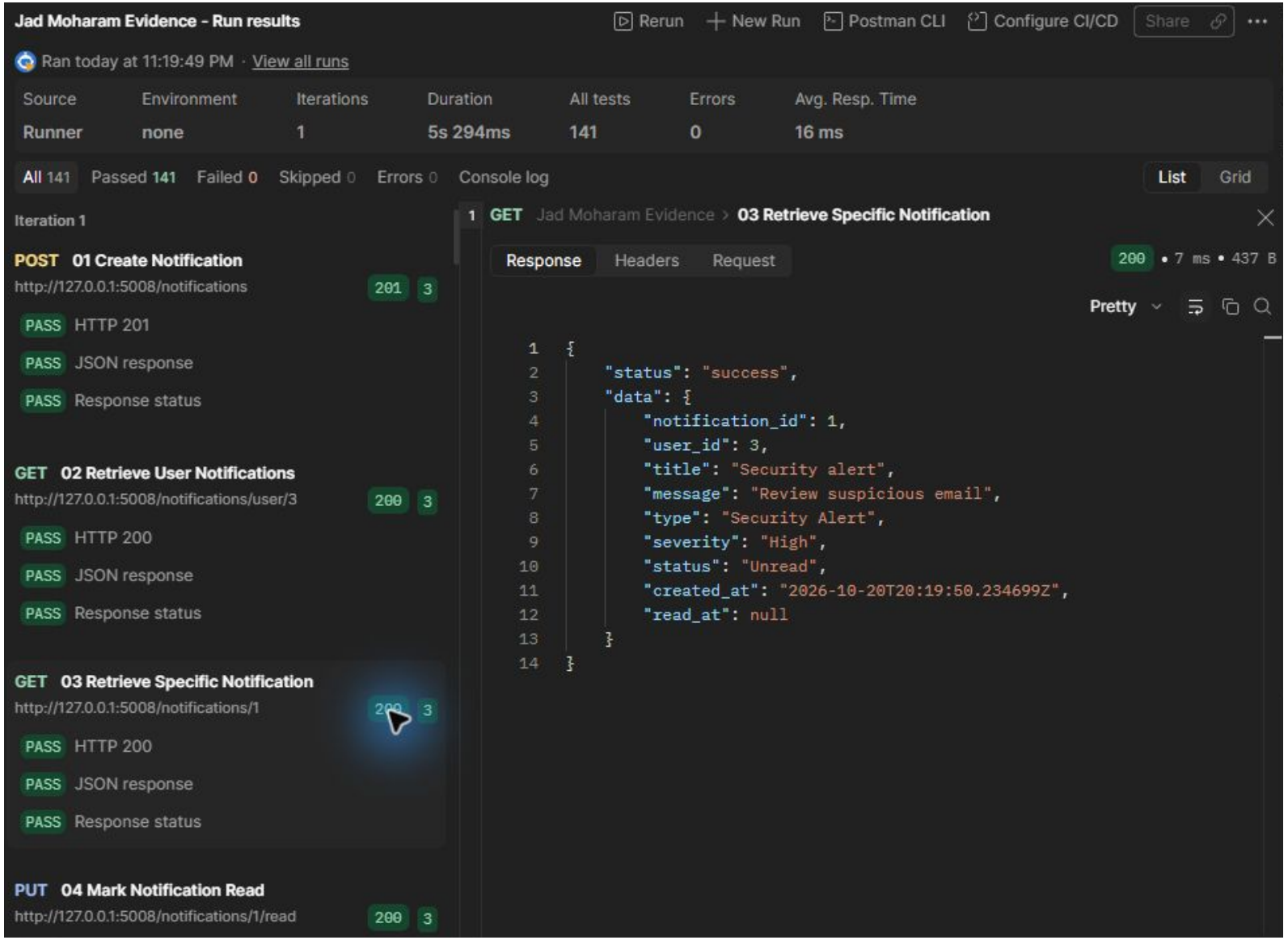


Retrieve Specific Notification

Jad Moharam

GET {{notifications_url}}/notifications/{{notification_id}}

Request JSON: No request body

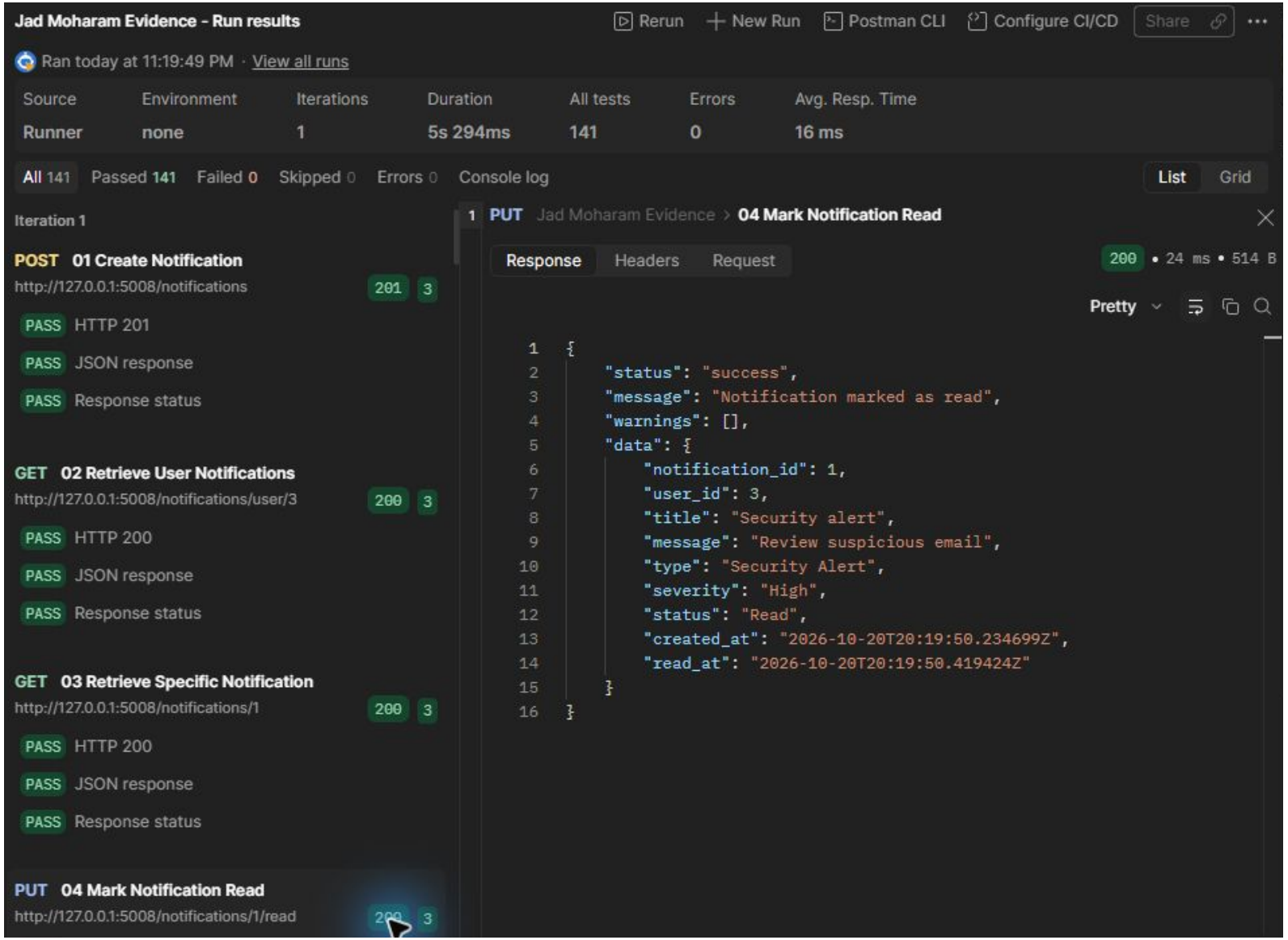


Mark Notification Read

Jad Moharam

PUT {{notifications_url}}/notifications/{{notification_id}}/read

Request JSON: No request body

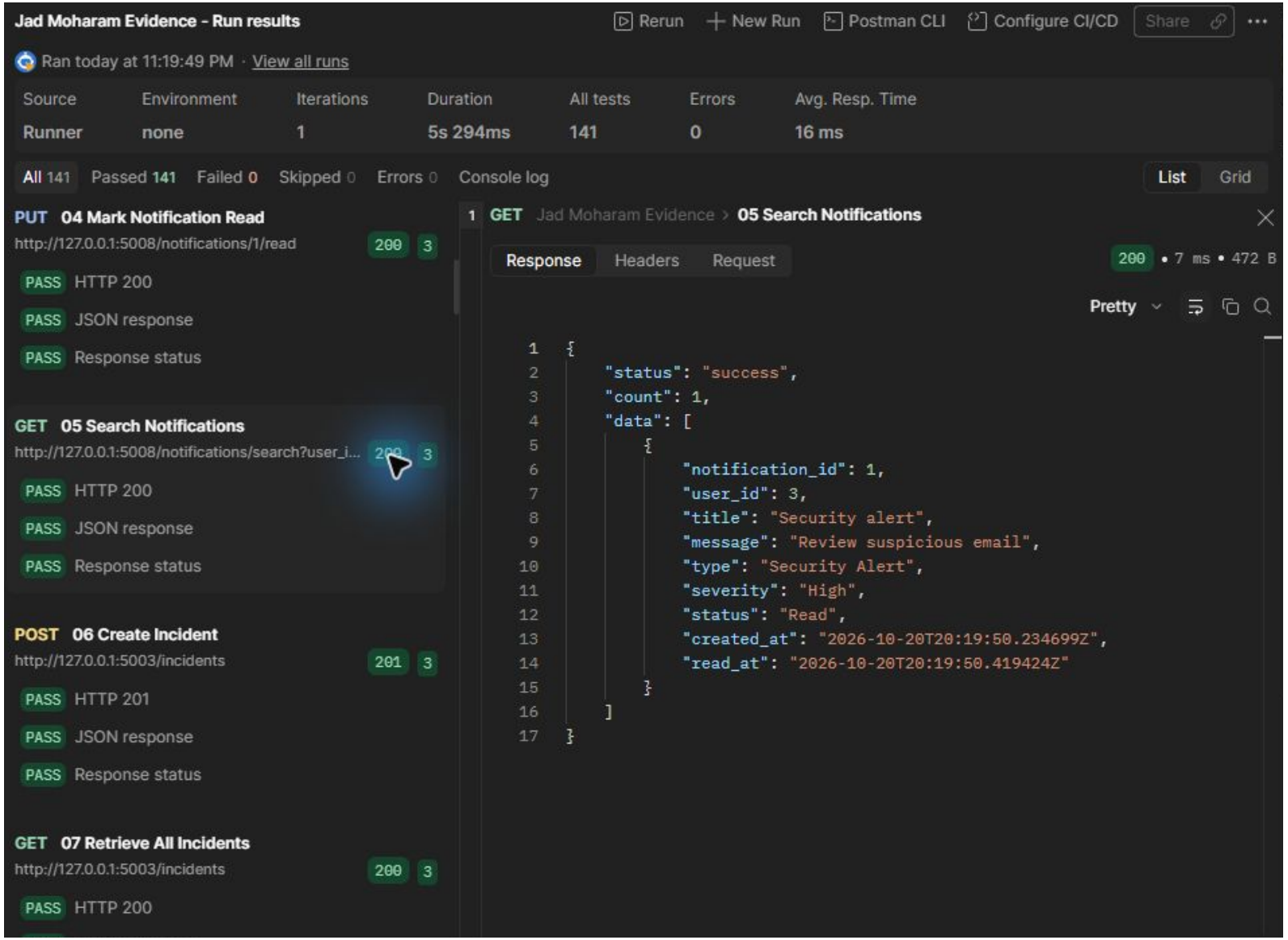


Search Notifications

Jad Moharam

GET {{notifications_url}}/notifications/search?user_id=3&type=Security%20Alert&severity=High&status=Read

Request JSON: No request body

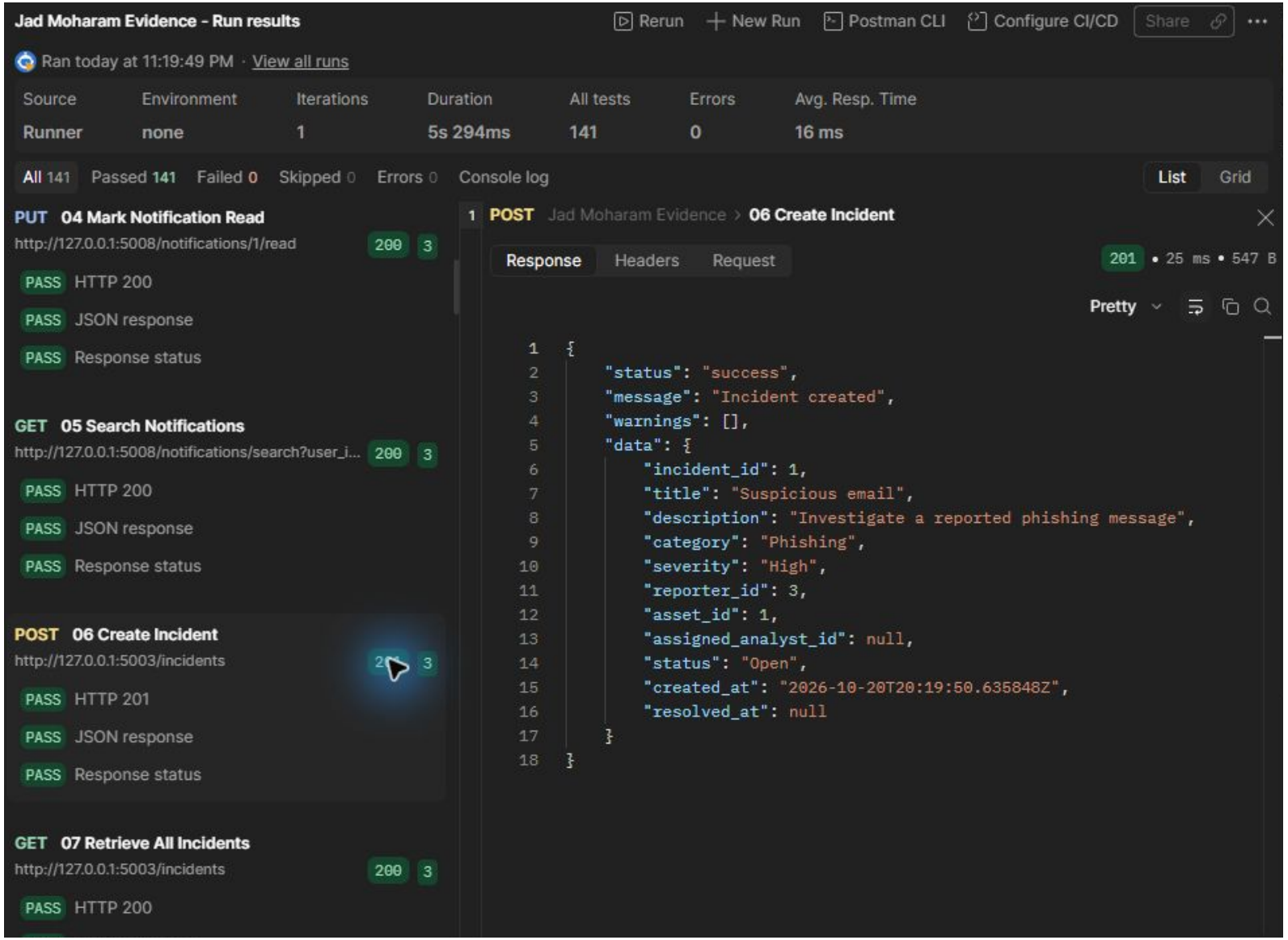


Create Incident

Jad Moharam

POST {{incidents_url}}/incidents

Request JSON: {"title":"Suspicious email","description":"Investigate a reported phishing message","category":"Phishing","severity":"High","reporter_id":3,"asset_id":1}

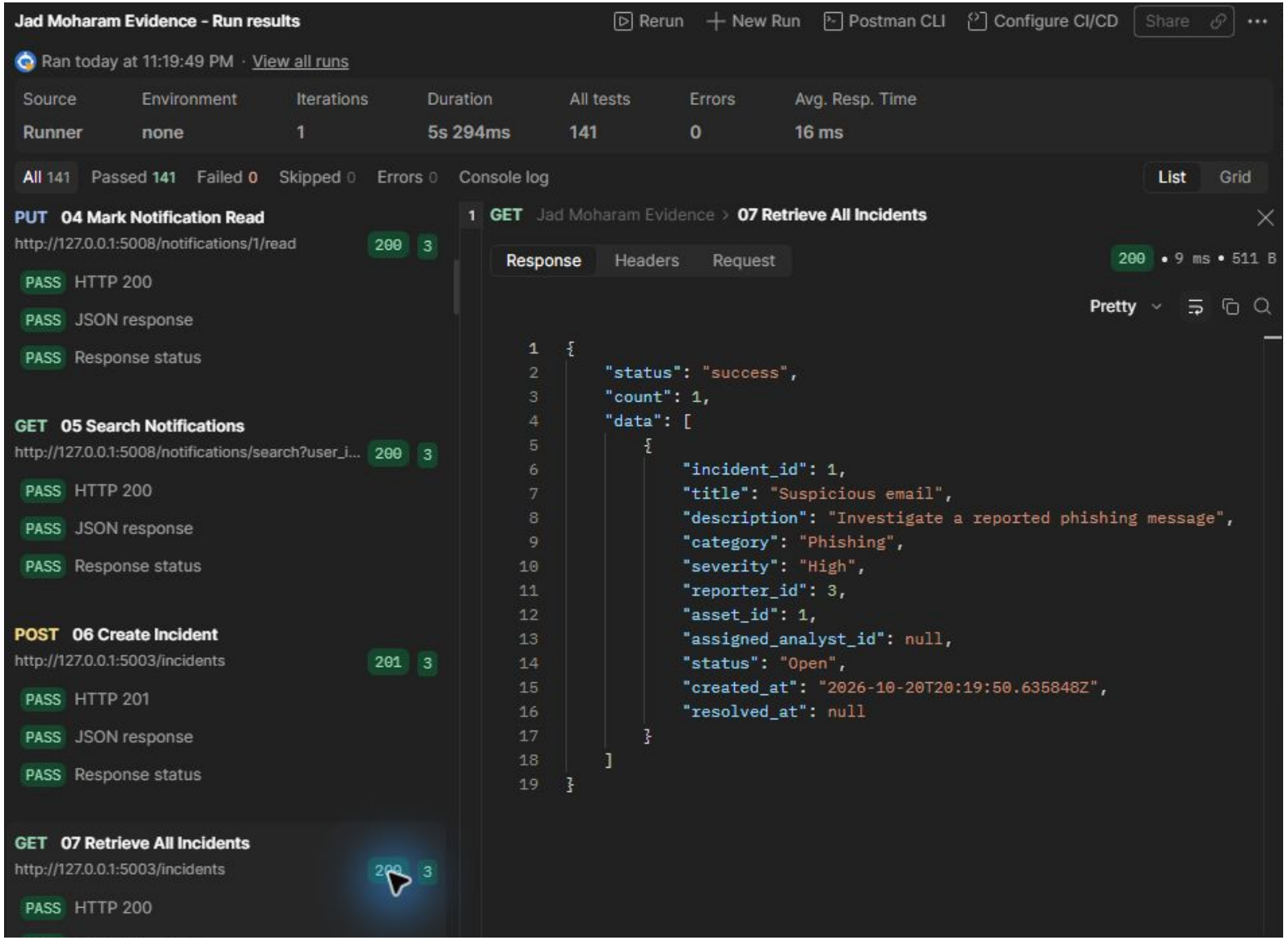


Retrieve All Incidents

Jad Moharam

GET {{incidents_url}}/incidents

Request JSON: No request body

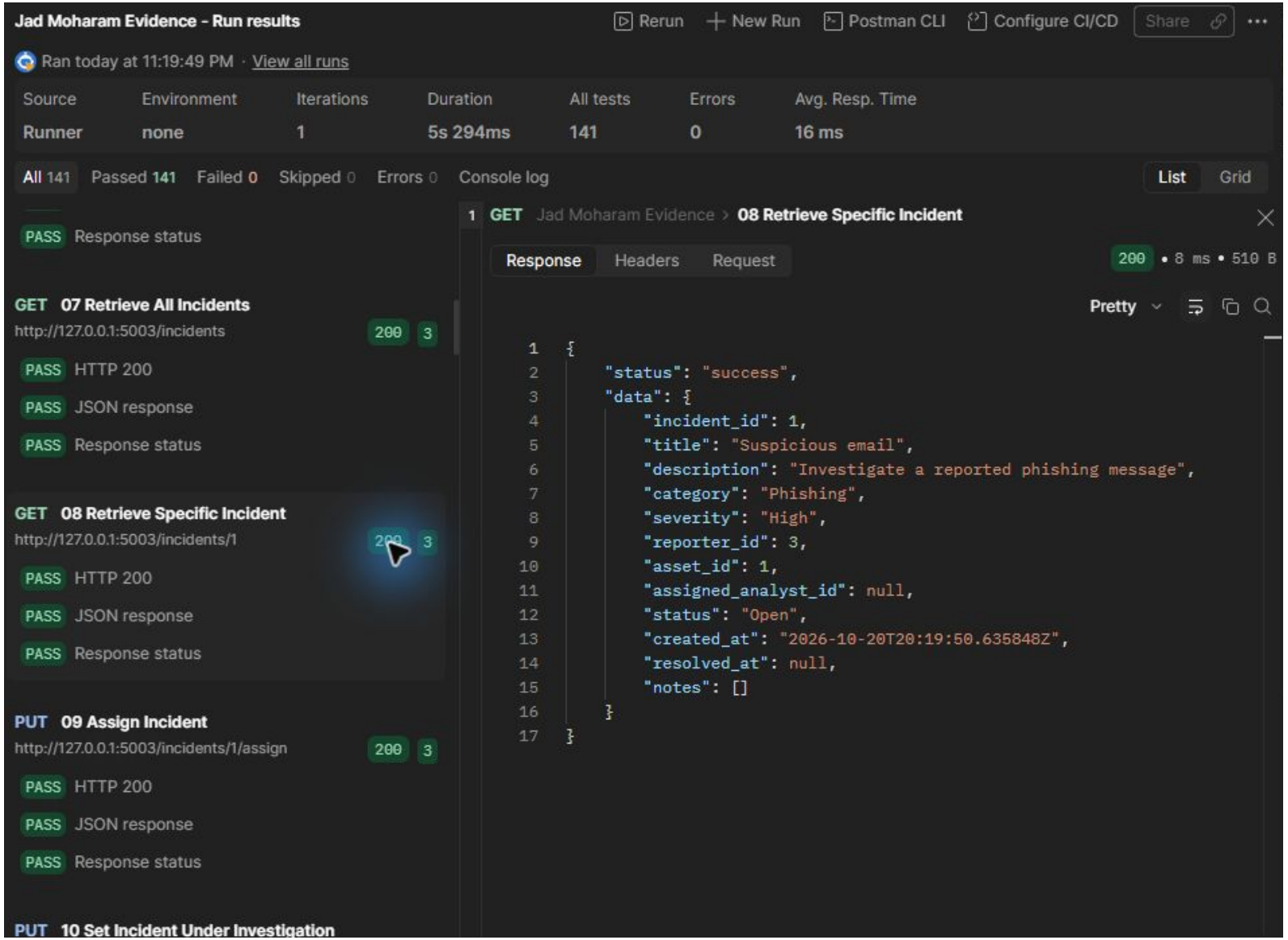


Retrieve Specific Incident

Jad Moharam

GET {{incidents_url}}/incidents/{{incident_id}}

Request JSON: No request body

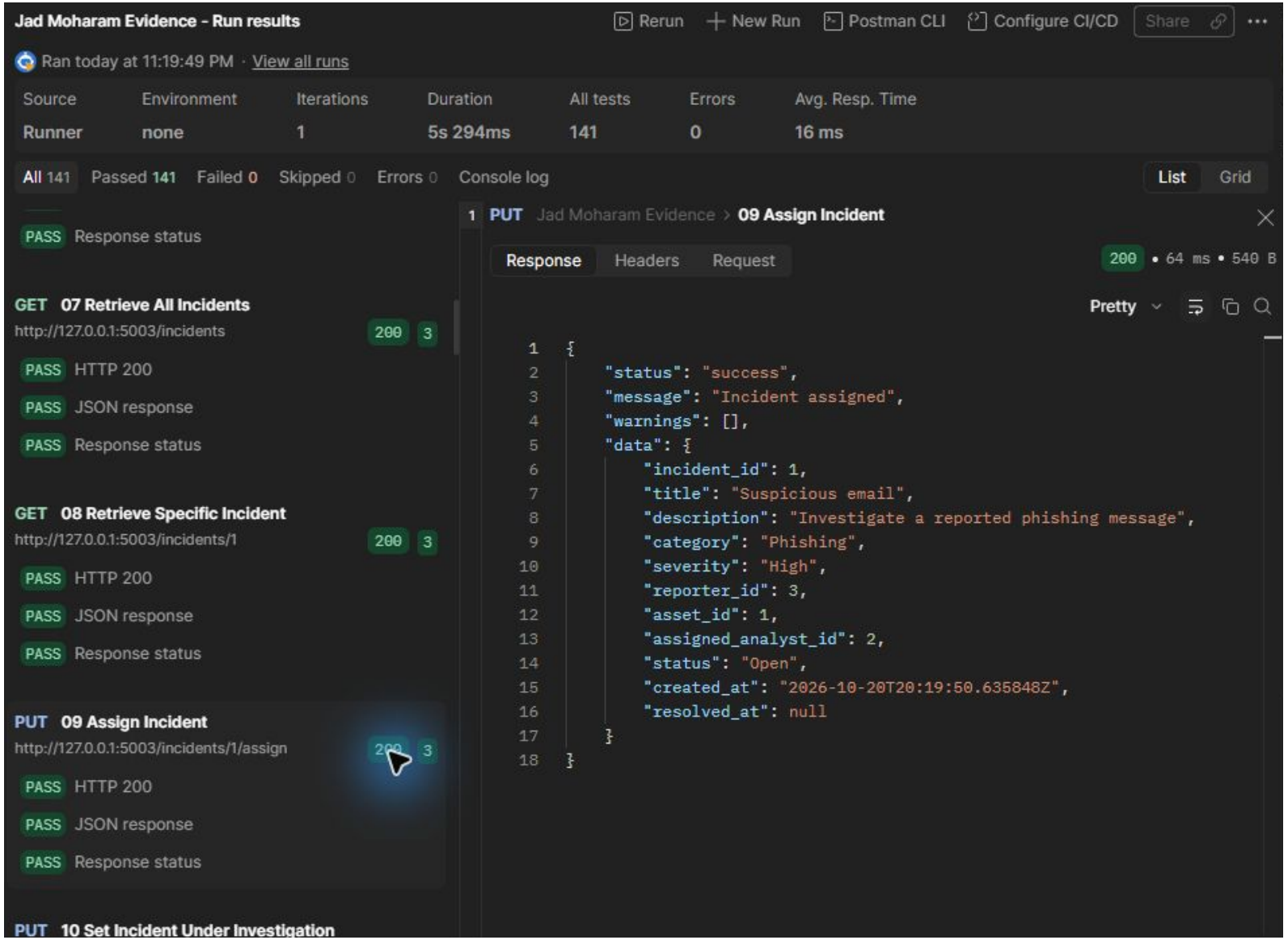


Assign Incident

Jad Moharam

PUT {{incidents_url}}/incidents/{{incident_id}}/assign

Request JSON: {"analyst_id":2}

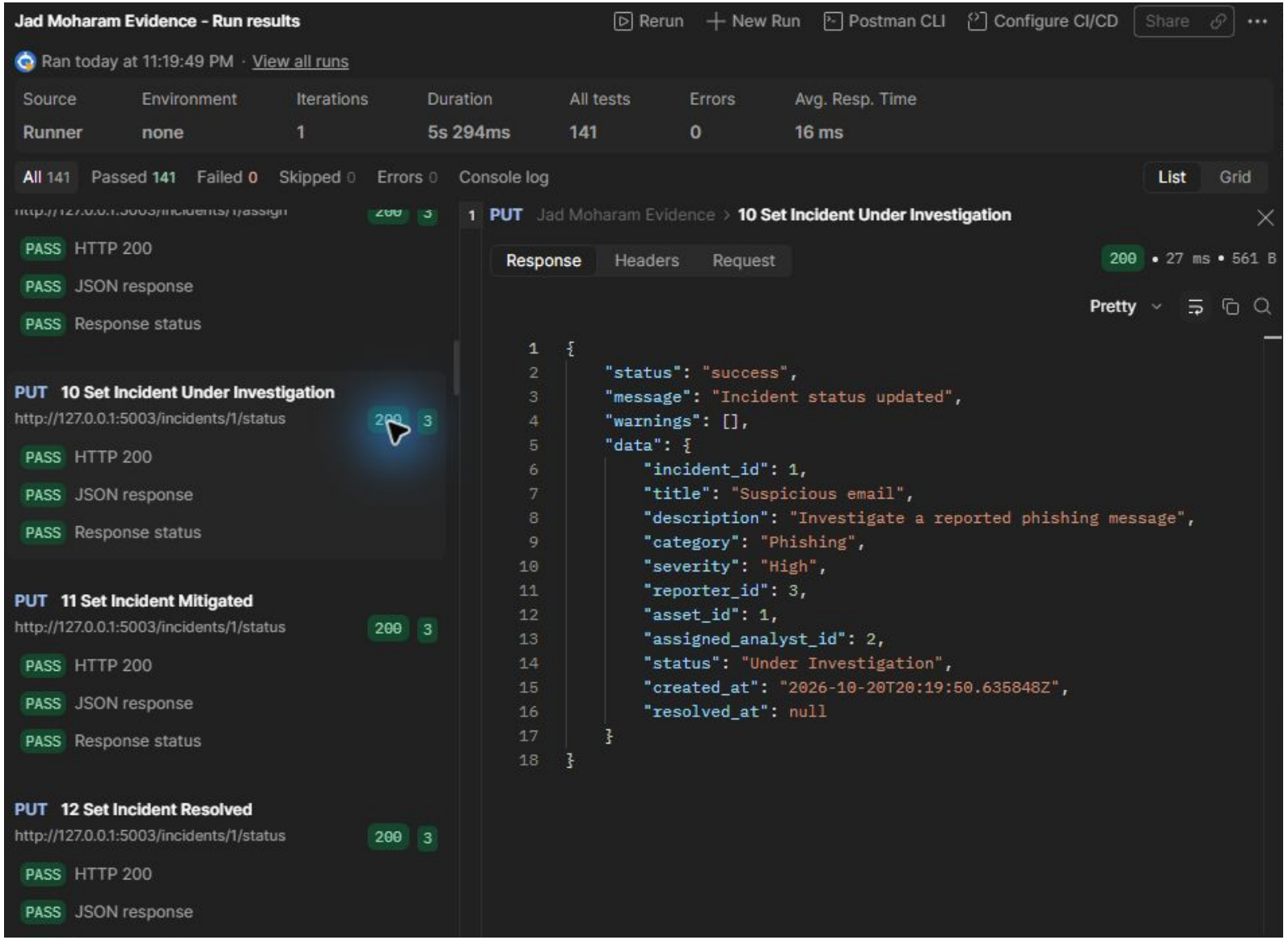


Set Incident Under Investigation

Jad Moharam

PUT {{incidents_url}}/incidents/{{incident_id}}/status

Request JSON: {"status":"Under Investigation"}

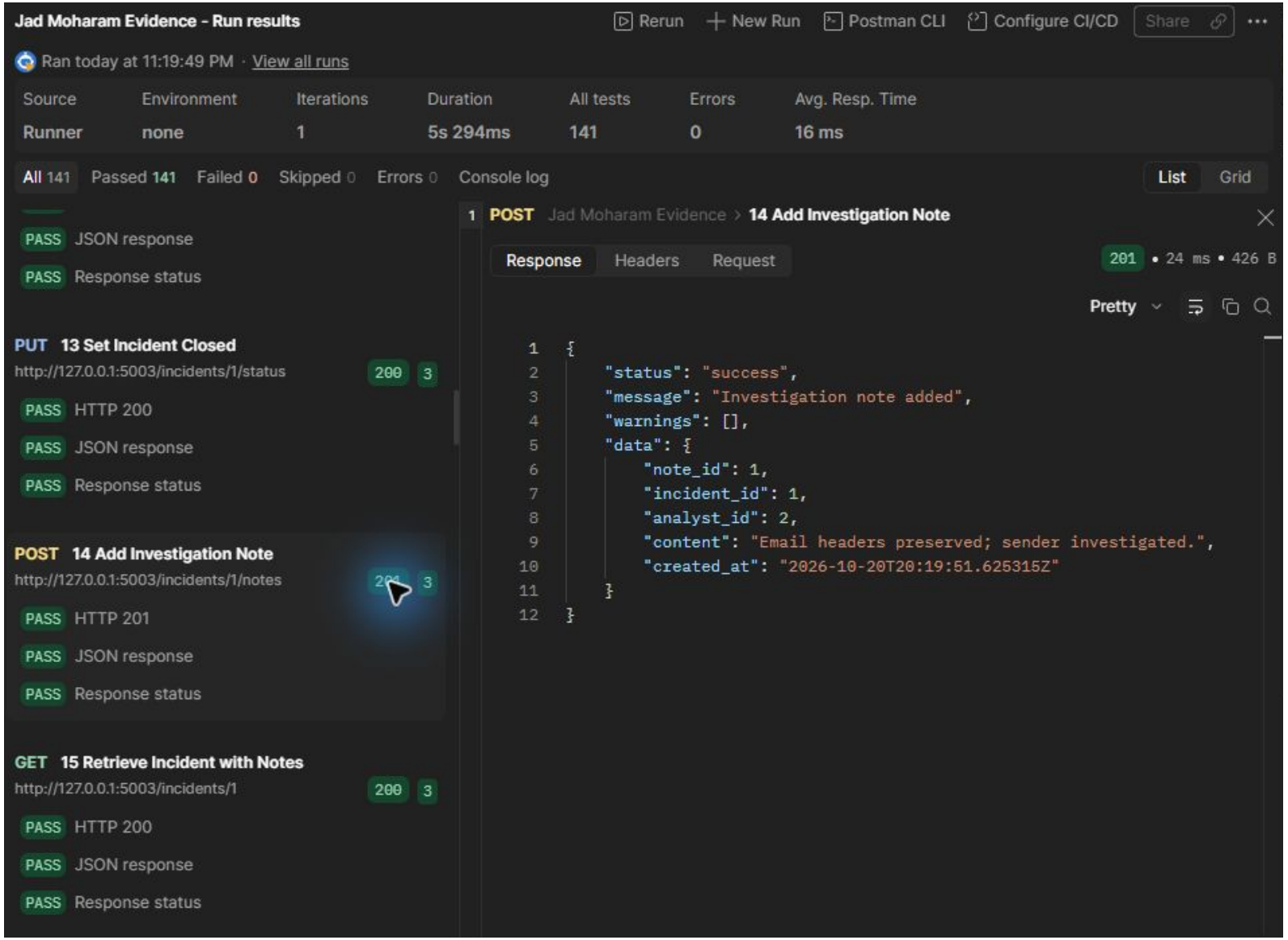


Add Investigation Note

Jad Moharam

POST {{incidents_url}}/incidents/{{incident_id}}/notes

Request JSON: {"content":"Email headers preserved; sender investigated."}

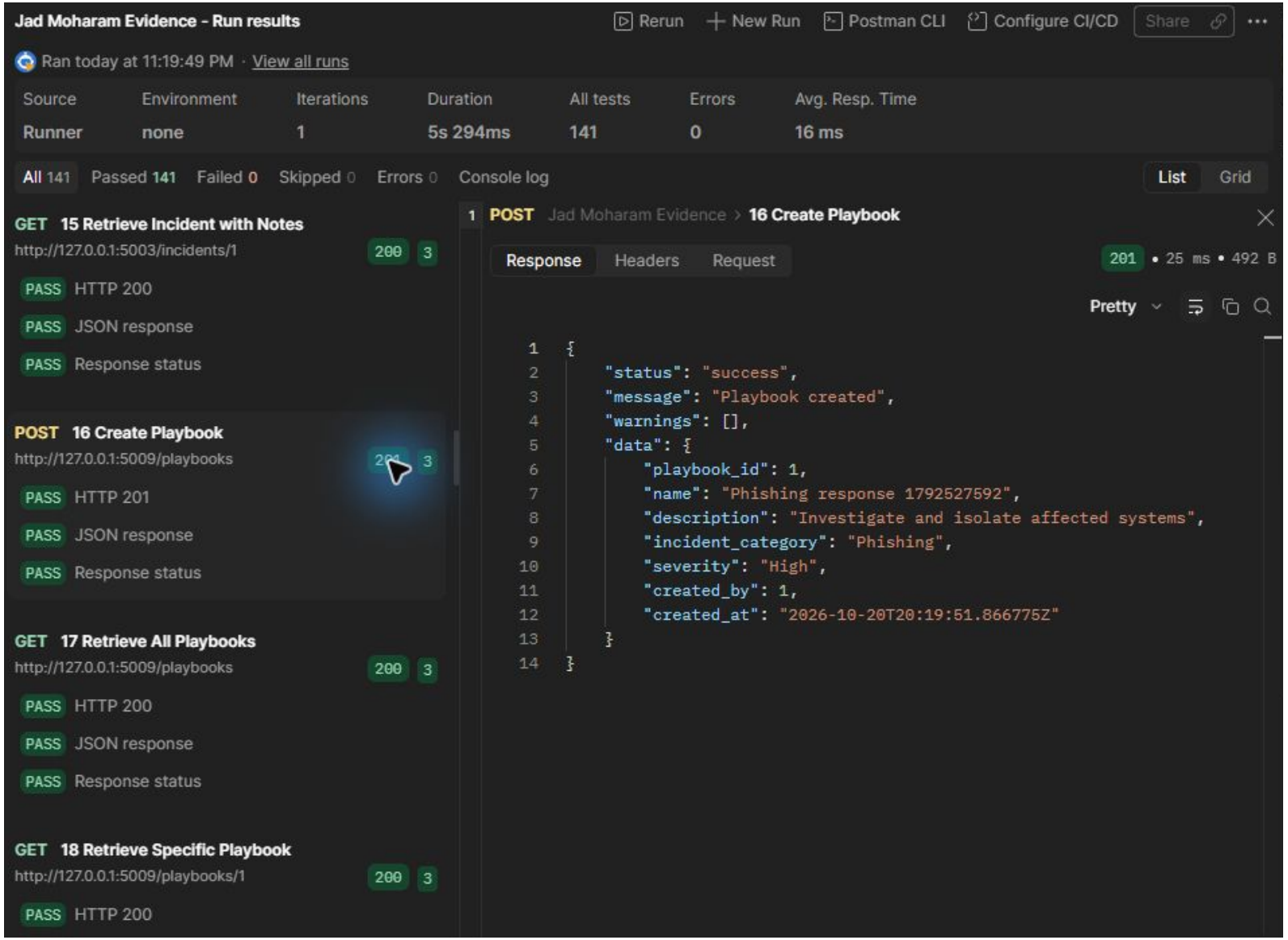


Create Playbook

Jad Moharam

POST {{playbooks_url}}/playbooks

Request JSON: {"name":"Phishing response {{$timestamp}}","description":"Investigate and isolate affected systems","incident_category":"Phishing","severity":"High"}

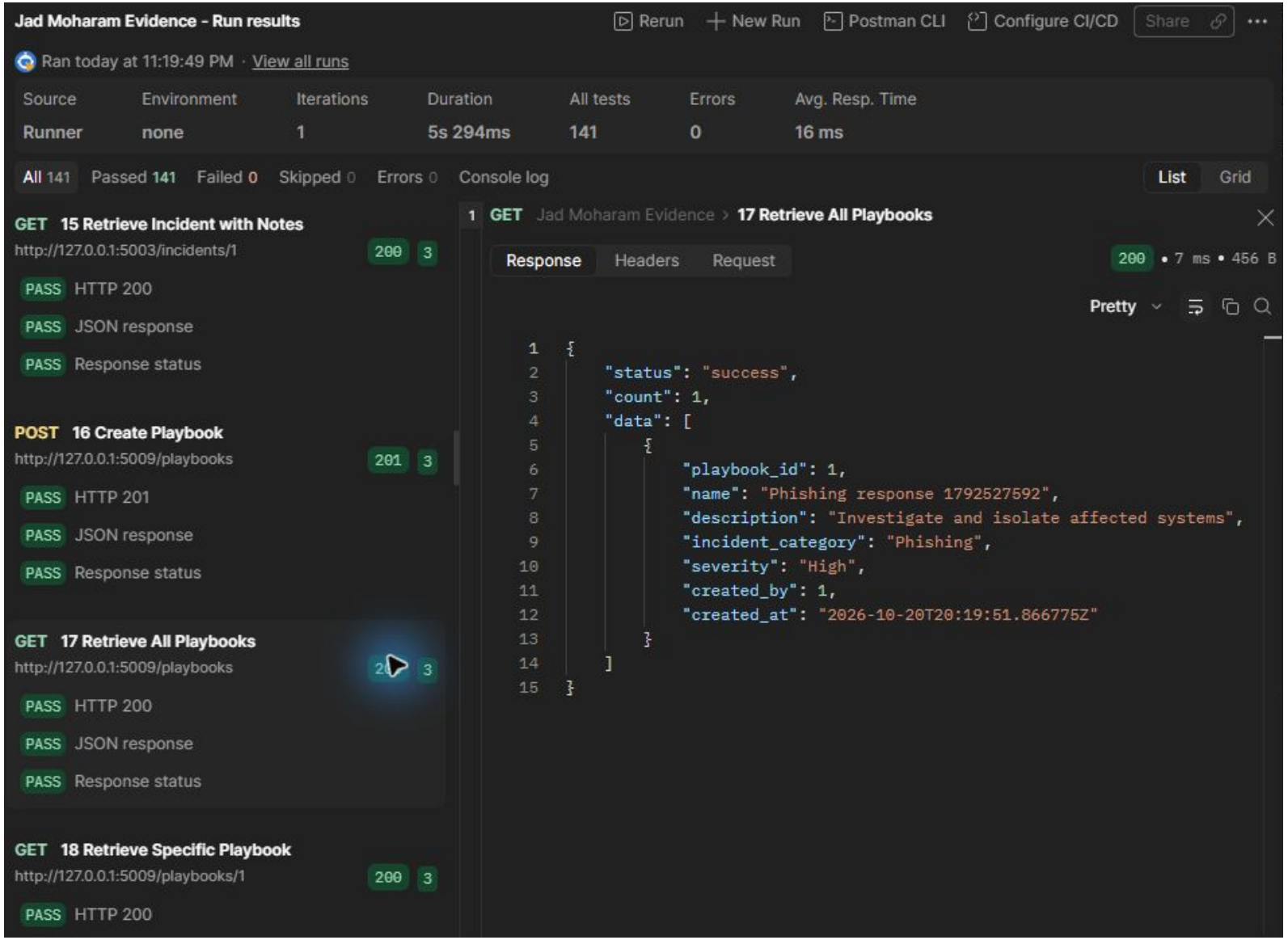


Retrieve All Playbooks

Jad Moharam

GET {{playbooks_url}}/playbooks

Request JSON: No request body

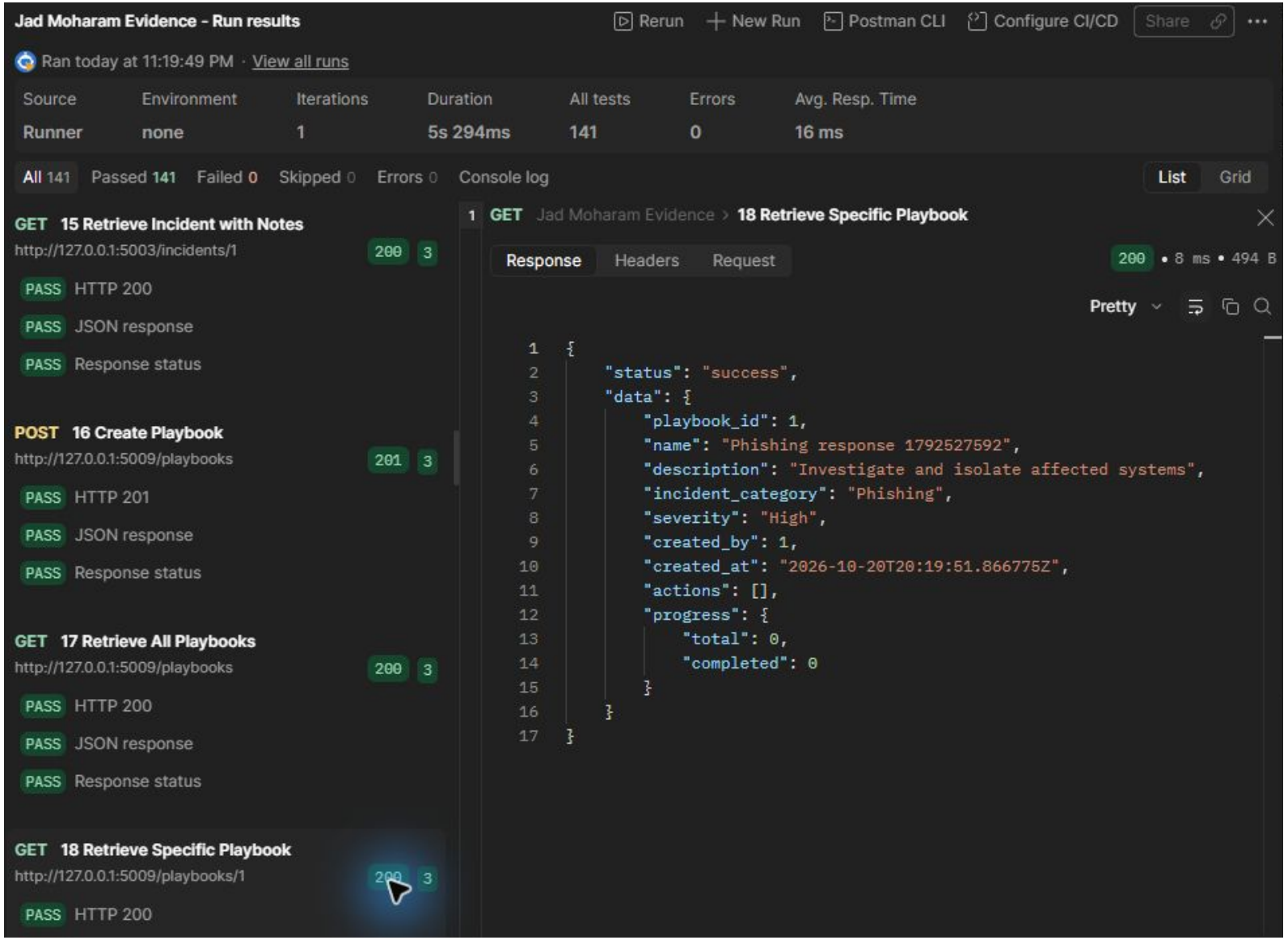


Retrieve Specific Playbook

Jad Moharam

GET {{playbooks_url}}/playbooks/{{playbook_id}}

Request JSON: No request body

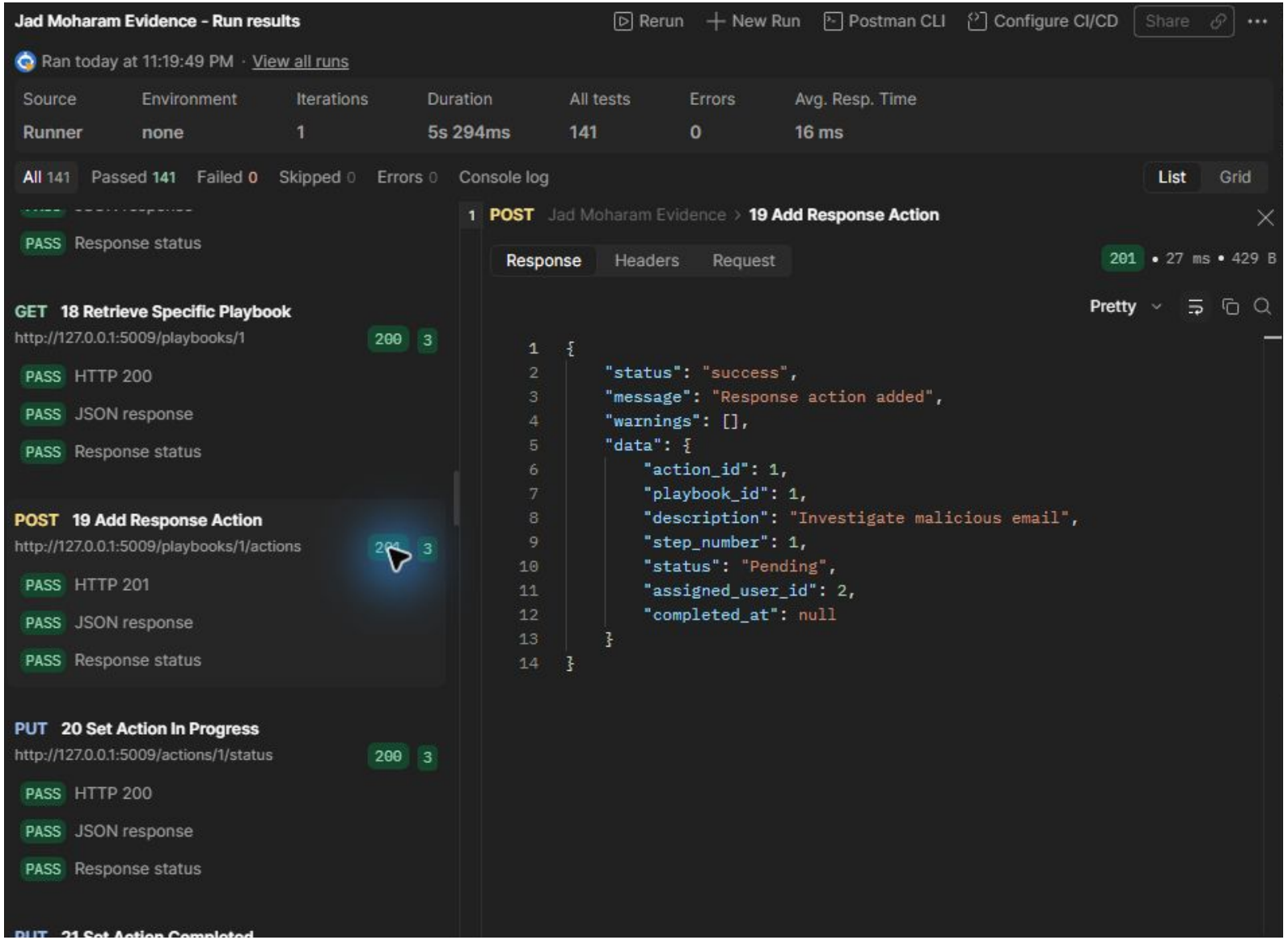


Add Response Action

Jad Moharam

POST {{playbooks_url}}/playbooks/{{playbook_id}}/actions

Request JSON: {"description":"Investigate malicious email","step_number":1,"assigned_user_id":2}

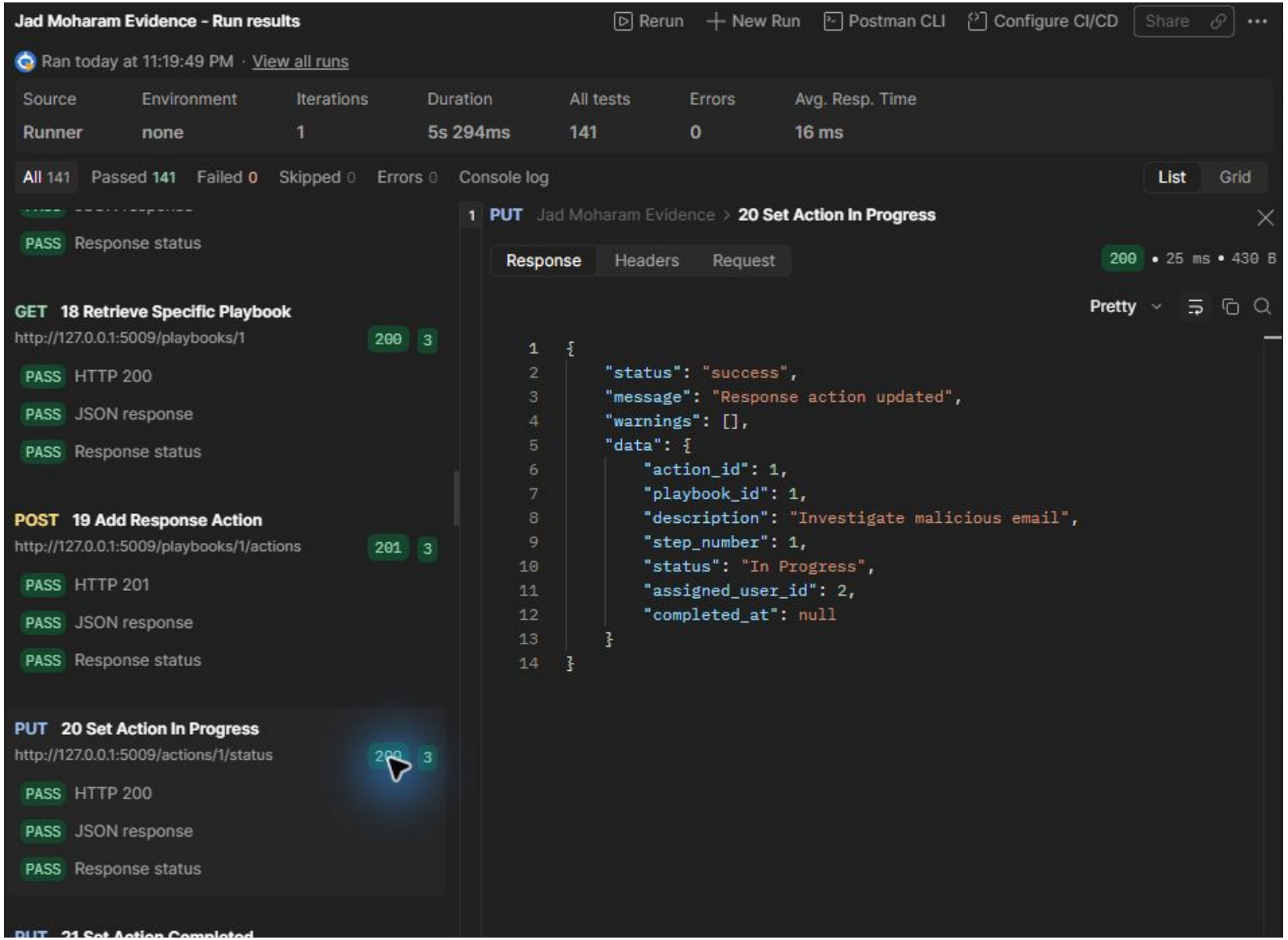


Set Action In Progress

Jad Moharam

PUT {{playbooks_url}}/actions/{{action_id}}/status

Request JSON: {"status":"In Progress"}

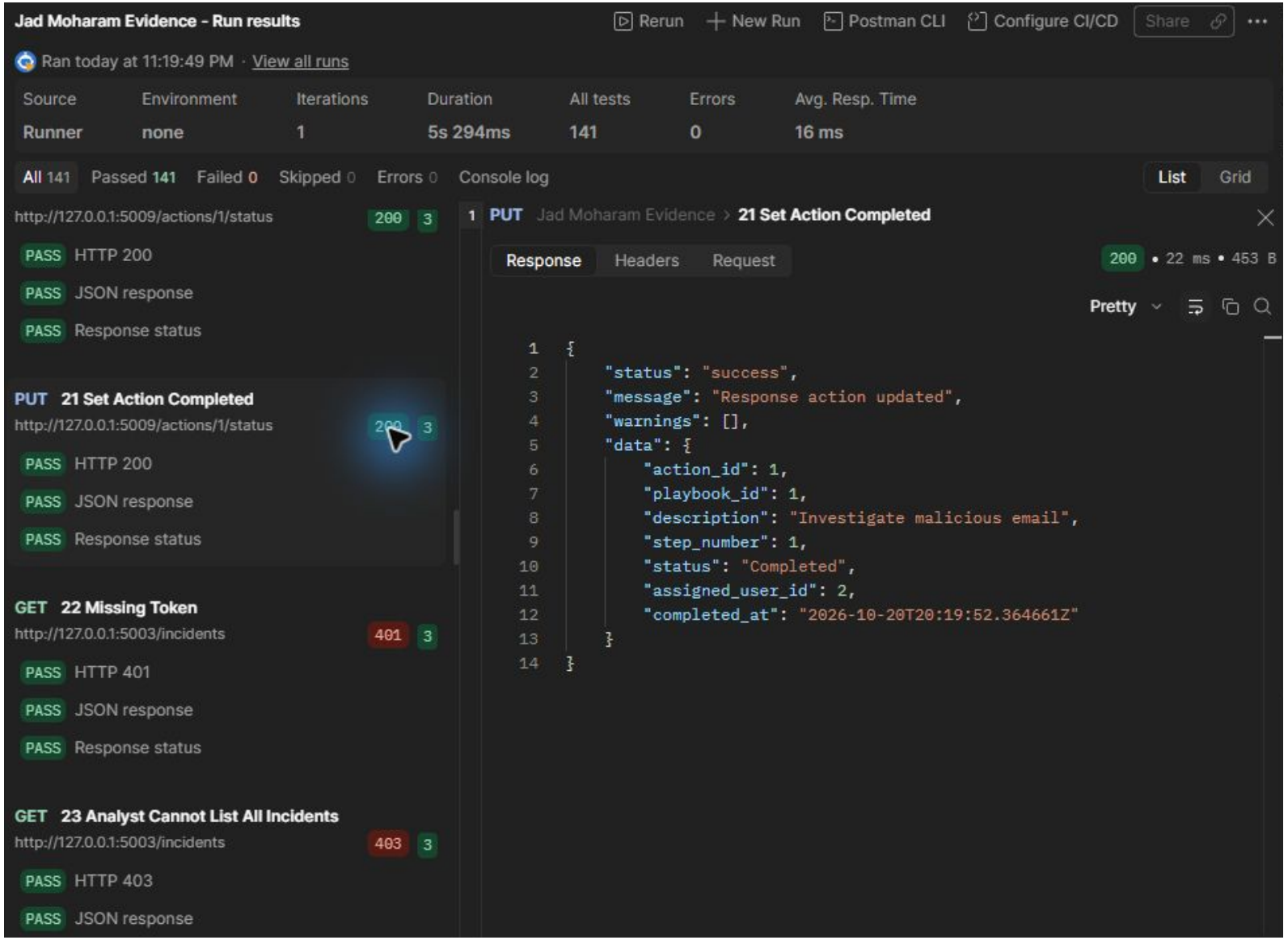


Set Action Completed

Jad Moharam

PUT {{playbooks_url}}/actions/{{action_id}}/status

Request JSON: {"status":"Completed"}

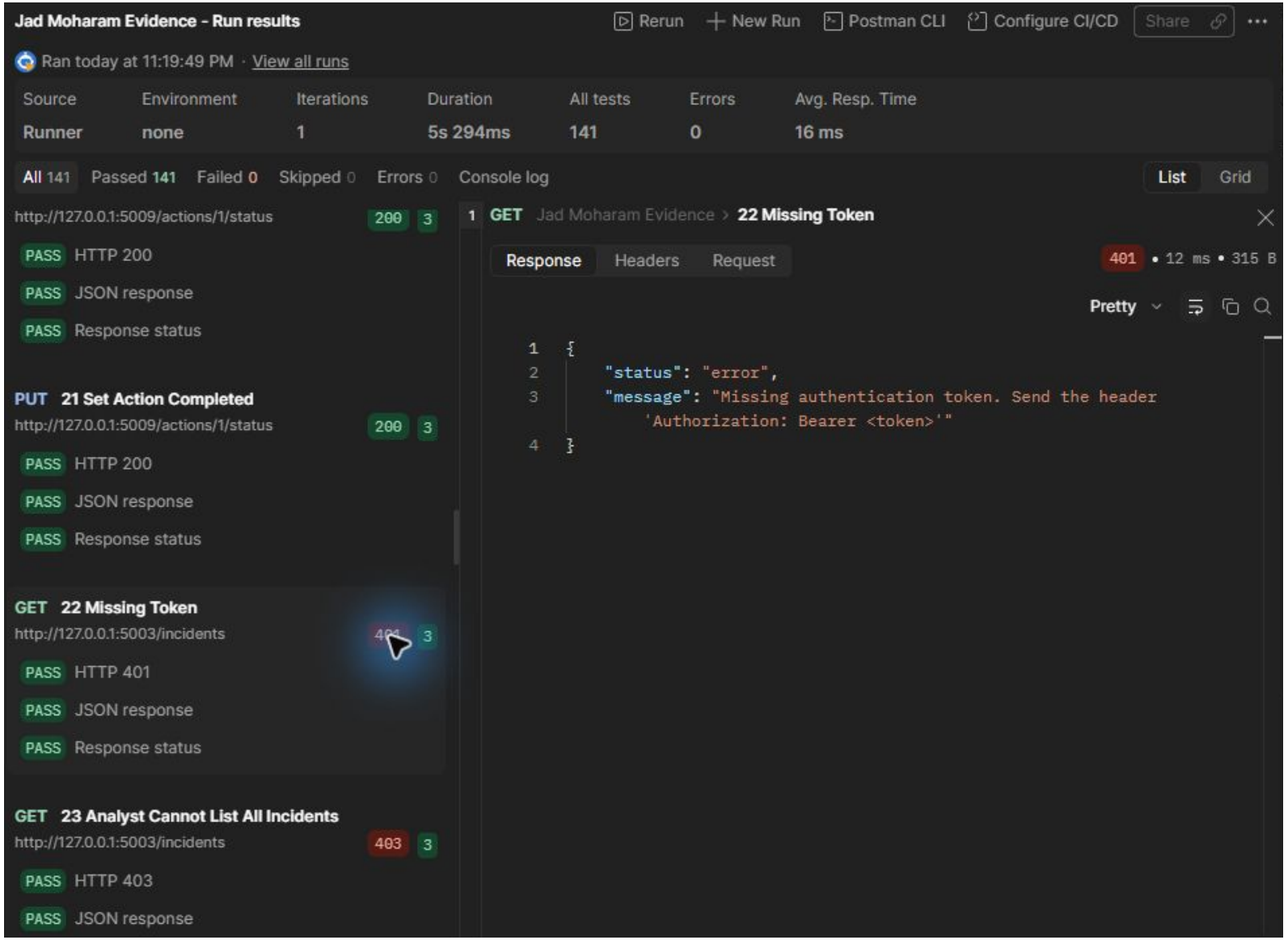


Missing Token

Jad Moharam

GET {{incidents_url}}/incidents

Request JSON: No request body

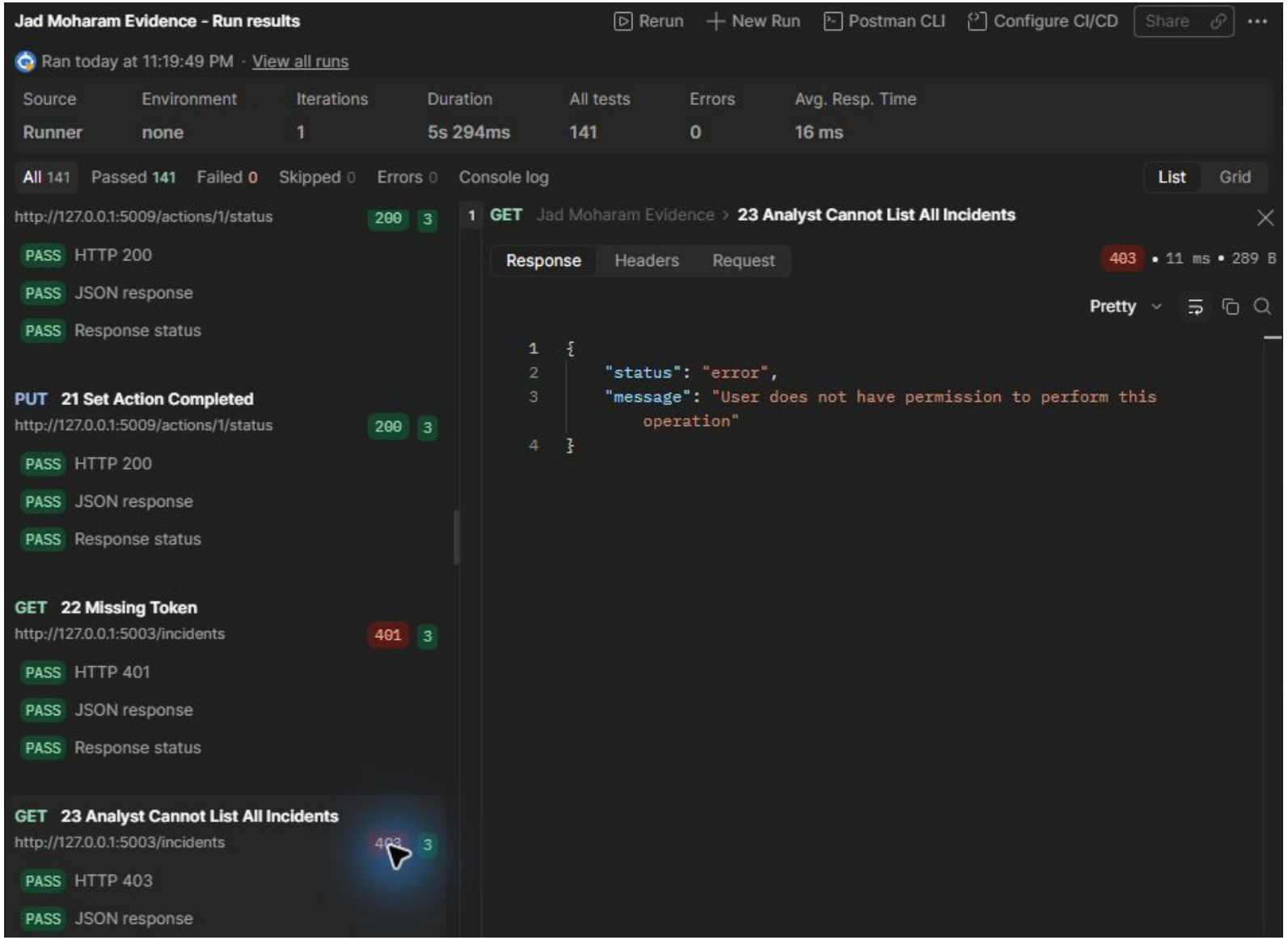


Analyst Cannot List All Incidents

Jad Moharam

GET {{incidents_url}}/incidents

Request JSON: No request body

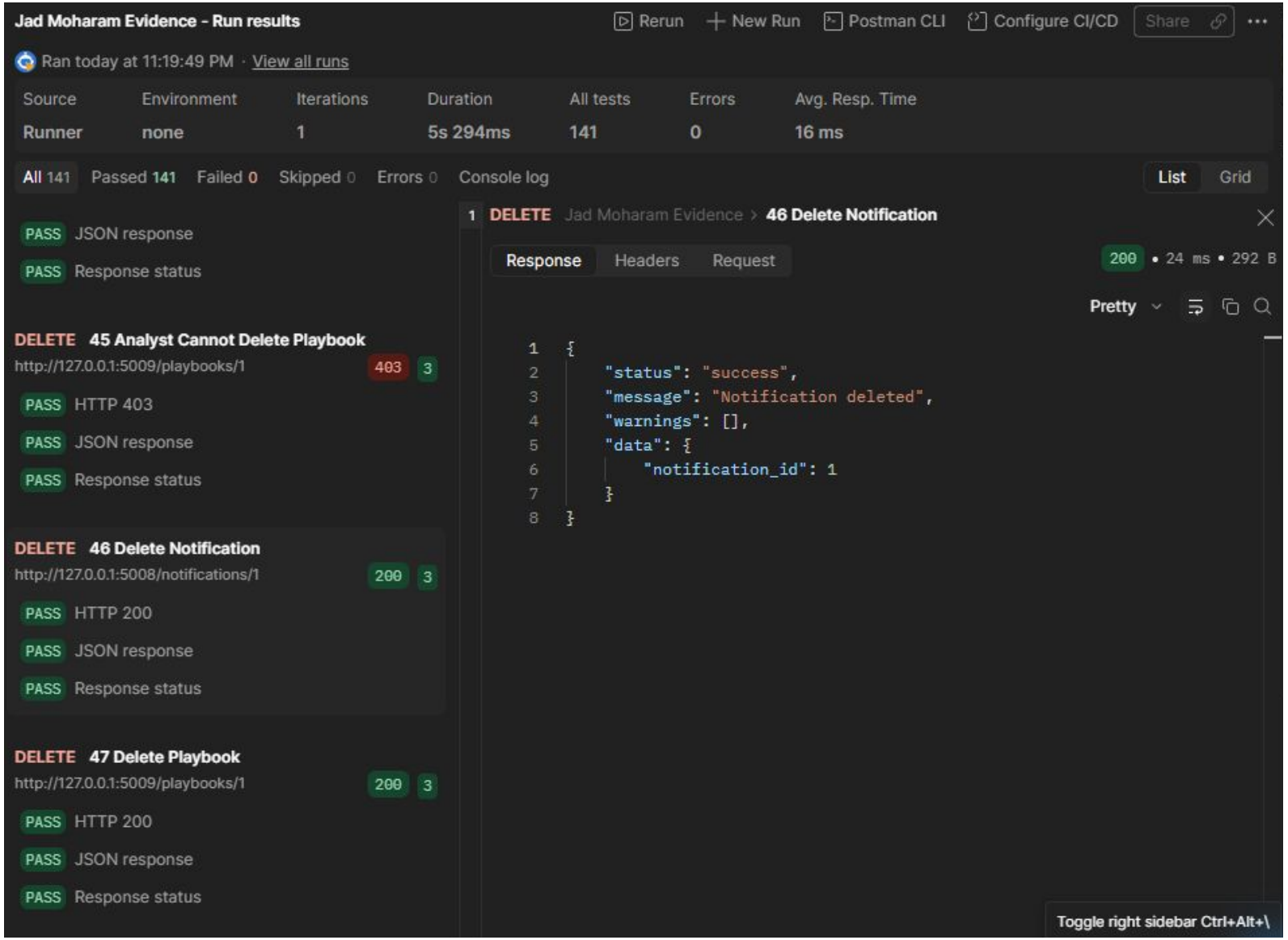


Delete Notification

Jad Moharam

DELETE {{notifications_url}}/notifications/{{notification_id}}

Request JSON: No request body

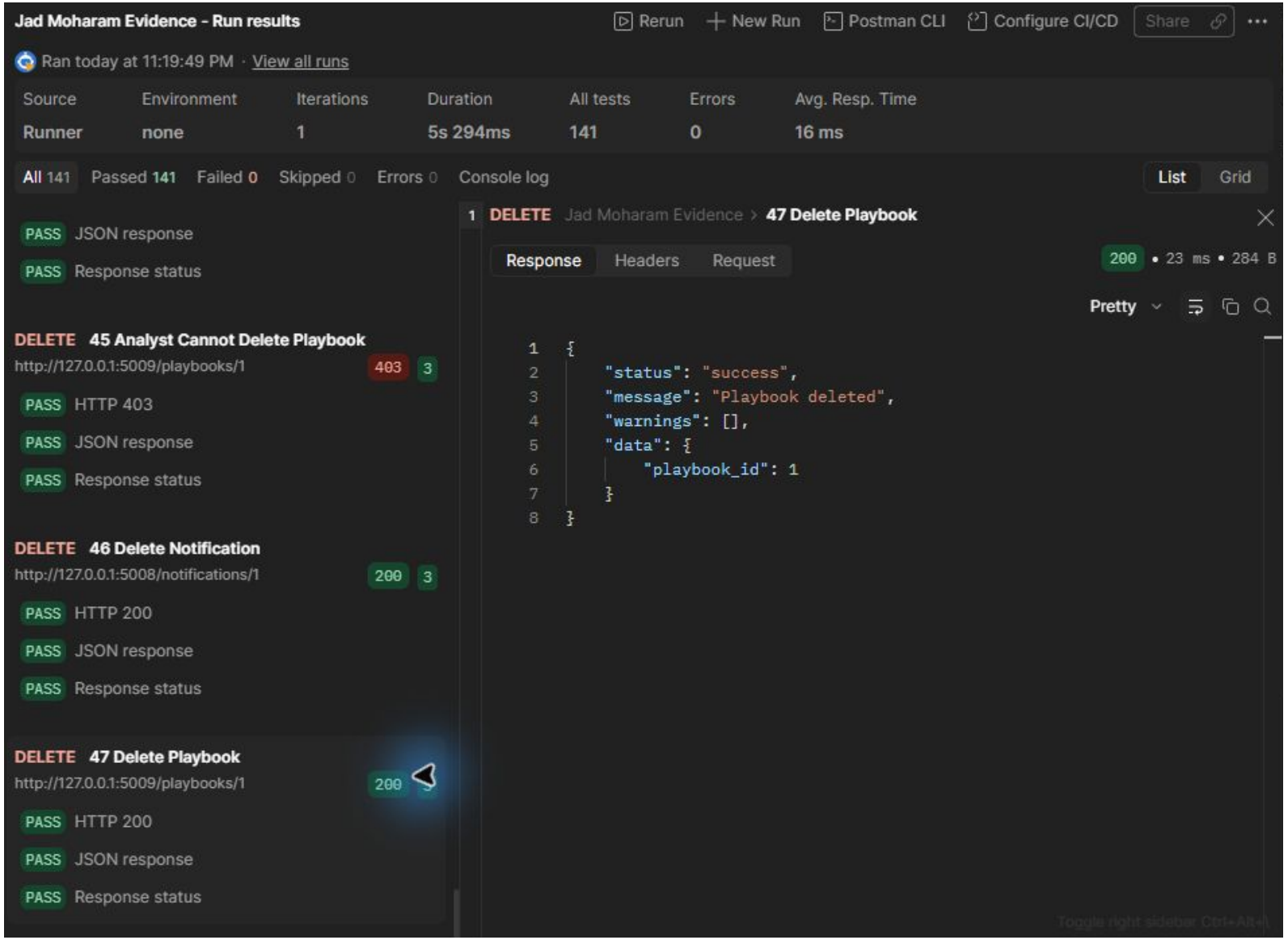


Delete Playbook

Jad Moharam

DELETE {{playbooks_url}}/playbooks/{{playbook_id}}

Request JSON: No request body



Database Baseline Before Requests

Jad Moharam

Jad Moharam Database Evidence

Real SQLite backup from the running local services

incidents

Before any API requests

Snapshot: 000-before-requests.db | Actual stored rows: 0

incidents | investigation_notes | notifications | playbooks | response_actions | action_history | logs | users | assets | jad_outbox

incident_id	title	description	category	severity	reporter_id	asset_id	assigned_analyst_id	status	created_at	resolved_at

Stored table definition

```
CREATE TABLE incidents (
 incident_id INTEGER PRIMARY KEY AUTOINCREMENT,
 title TEXT NOT NULL, description TEXT NOT NULL, category TEXT NOT
NULL,
 severity TEXT NOT NULL CHECK(severity IN
('Low','Medium','High','Critical')),
 reporter_id INTEGER NOT NULL REFERENCES users(user_id),
 asset_id INTEGER REFERENCES assets(asset_id),
 assigned_analyst_id INTEGER REFERENCES users(user_id),
 status TEXT NOT NULL DEFAULT 'Open' CHECK(status IN
('Open','Under Investigation','Mitigated','Resolved','Closed')),
 created_at TEXT NOT NULL, resolved_at TEXT
)
```

Separate development fixtures supply Users and Assets. Their production services are not part of this evidence run.

Closed Incident Stored in SQLite

Jad Moharam

Jad Moharam Database Evidence

incidents

Actual SQLite snapshot 016-incidents.db | Displayed records 1

Stored record 1

incident_id	1
title	Suspicious email
description	Investigate a reported phishing message
category	Phishing
severity	High
reporter_id	3
asset_id	1
assigned_analyst_id	2
status	Closed
created_at	2026-10-20T20:19:50.635848Z
resolved_at	2026-10-20T20:19:51.415751Z

Read only view of actual backups. Users and Assets are isolated development fixtures.

Investigation Note Stored in SQLite

Jad Moharam

Jad Moharam Database Evidence

investigation_notes

Actual SQLite snapshot 016-incidents.db | Displayed records 1

Stored record 1

note_id	1
incident_id	1
analyst_id	2
content	Email headers preserved; sender investigated.
created_at	2026-10-20T20:19:51.625315Z

Read only view of actual backups. Users and Assets are isolated development fixtures.

Completed Response Action Stored in SQLite

Jad Moharam

Jad Moharam Database Evidence

response_actions

Actual SQLite snapshot 022-playbooks.db | Displayed records 1

Stored record 1

action_id	1
playbook_id	1
description	Investigate malicious email
step_number	1
status	Completed
assigned_user_id	2
completed_at	2026-10-20T20:19:52.364661Z

Read only view of actual backups. Users and Assets are isolated development fixtures.

Playbook Removed After Authorized Deletion

Jad Moharam



Add Threat Indicator

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 201. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Add Threat Indicator

Actual HTTP execution of the Flask implementation

POST /threats

HTTP 201 | Expected 201 | 2026-10-20T20:32:11.750562+00:00

Request JSON


```
{
  "threat_type": "Domain",
  "threat_value": "sprint-evidence.example",
  "description": "Development evidence indicator",
  "severity": "High",
  "source": "Team local test"
}
```


Response JSON


```
{
  "status": "success",
  "message": "Threat indicator added successfully",
  "data": {
    "threat_id": 1,
    "threat_type": "Domain",
    "threat_value": "sprint-evidence.example",
    "description": "Development evidence indicator",
    "severity": "High",
    "source": "Team local test",
    "date_added": "2026-10-20 20:32:11",
    "last_updated": null
  }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Update Threat Information

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 200. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Update Threat Information

Actual HTTP execution of the Flask implementation

PUT /threats/1

HTTP 200 | Expected 200 | 2026-10-20T20:32:11.771008+00:00

Request JSON


```
{
  "severity": "Critical",
  "description": "Confirmed development indicator"
}
```


Response JSON


```
{
  "status": "success",
  "message": "Threat information updated successfully",
  "previous_values": {
    "description": "Development evidence indicator",
    "severity": "High"
  },
  "data": {
    "threat_id": 1,
    "threat_type": "Domain",
    "threat_value": "sprint-evidence.example",
    "description": "Confirmed development indicator",
    "severity": "Critical",
    "source": "Team local test",
    "date_added": "2026-10-20 20:32:11",
    "last_updated": "2026-10-20 20:32:11"
  }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Create Log Record

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 201. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Create Log Record

Actual HTTP execution of the Flask implementation

POST /logs

HTTP 201 | Expected 201 | 2026-10-20T20:32:11.777010+00:00

Request JSON


```
{
  "user_id": 1,
  "action": "TEAM_EVIDENCE",
  "description": "Common report evidence run",
  "severity": "Information",
  "related_service": "Users"
}
```


Response JSON


```
{
  "status": "success",
  "message": "Log record created successfully",
  "data": {
    "log_id": 30,
    "user_id": 1,
    "action": "TEAM_EVIDENCE",
    "description": "Common report evidence run",
    "timestamp": "2026-10-20 20:32:11",
    "severity": "Information",
    "related_service": "Users"
  }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Summary Report

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 503. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Summary Report

Actual HTTP execution of the Flask implementation

GET /reports/summary

HTTP 503 | Expected 503 | 2026-10-20T20:32:13.841525+00:00

Request JSON

```
null
```

Response JSON

```
{
  "status": "error",
  "message": "Could not generate the Security Summary Report: The Users
and Identity Management service is not reachable at
http://127.0.0.1:5191/users. Make sure it is running",
  "errors": {
    "source": "Users and Identity Management"
  }
}
```

Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Incidents Report

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 200. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Incidents Report

Actual HTTP execution of the Flask implementation

GET /reports/incidents

HTTP 200 | Expected 200 | 2026-10-20T20:32:13.847524+00:00

Request JSON

```
null
```

Response JSON


```
{
  "status": "success",
  "message": "Incident Report generated successfully",
  "data": {
    "total_incidents": 1,
    "incidents_by_severity": {
      "Low": 0,
      "Medium": 0,
      "High": 1,
      "Critical": 0
    },
    "incidents_by_status": {
      "Open": 0,
      "Under Investigation": 0,
      "Mitigated": 0,
      "Resolved": 0,
      "Closed": 1
    },
    "generated_at": "2026-10-20 20:32:13 UTC",
    "data_sources": [
      "Incident Management service: GET
http://127.0.0.1:5103/incidents"
    ]
  }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Threats Report Continued 1

Omar Tabsh

Actual HTTP capture in consecutive sections. HTTP 200. Request 14.

Omar Tabsh Threats Report

Actual HTTP execution of the Flask implementation

GET /reports/threats

HTTP 200 | Expected 200 | 2026-10-20T20:32:15.875644+00:00

Request JSON

```
null
```

Response JSON


```
{
  "status": "success",
  "message": "Threat Report generated successfully",
  "data": {
    "total_threats": 1,
    "threats_by_severity": {
      "Low": 0,
      "Medium": 0,
      "High": 0,
      "Critical": 1
```

Threats Report Continued 2

Omar Tabsh

Actual HTTP capture in consecutive sections. HTTP 200. Request 14.


```
  },
  "threats_by_type": {
    "IP Address": 0,
    "Domain": 1,
    "File Hash": 0,
    "Malware Signature": 0,
    "Other": 0
  },
  "recently_added_threats": [
    {
      "threat_id": 1,
      "threat_type": "Domain",
      "threat_value": "sprint-evidence.example",
      "severity": "Critical",
      "source": "Team local test",
      "date_added": "2026-10-20 20:32:11"
    }
  ],
  "generated_at": "2026-10-20 20:32:15 UTC",
  "data_sources": [
    "Threat Intelligence service: GET http://127.0.0.1:5104/threats"
  ]
 }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Activity Report Continued 1

Omar Tabsh

Actual HTTP capture in consecutive sections. HTTP 200. Request 16.

Omar Tabsh Activity Report

Actual HTTP execution of the Flask implementation

GET /reports/activity

HTTP 200 | Expected 200 | 2026-10-20T20:32:17.930108+00:00

Request JSON

```
null
```

Response JSON


```
{
  "status": "success",
  "message": "Activity Report generated successfully",
  "data": {
    "total_user_activities": 30,
    "login_attempts": 0,
    "successful_logins": 0,
    "failed_login_attempts": 0,
    "unauthorized_access_attempts": 10,
    "most_performed_actions": [
      {
        "action": "UNAUTHORIZED_ACCESS",
        "count": 10
      },
      {
```

Activity Report Continued 2

Omar Tabsh

Actual HTTP capture in consecutive sections. HTTP 200. Request 16.


```
    {
      "action": "INCIDENT_STATUS_CHANGED",
      "count": 4
    },
    {
      "action": "NOTIFICATION_CREATED",
      "count": 2
    },
    {
      "action": "RESPONSE_ACTION_UPDATED",
      "count": 2
    },
    {
      "action": "INCIDENT_ASSIGNED",
      "count": 1
    }
  ],
  "activities_by_severity": {
    "Information": 20,
    "Warning": 10,
    "Error": 0,
    "Critical": 0
  },
  "generated_at": "2026-10-20 20:32:17 UTC",
  "data_sources": [
    "Security Audit Logging service: GET http://127.0.0.1:5105/logs"
  ]
 }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Reject Regular User Threat Creation

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 403. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Reject Regular User Threat Creation

Actual HTTP execution of the Flask implementation

POST /threats

HTTP 403 | Expected 403 | 2026-10-20T20:32:17.939887+00:00

Request JSON

```
{}
```

Response JSON

```
{
  "status": "error",
  "message": "User does not have permission to perform this operation"
}
```

Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Reject Invalid Domain

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 400. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Reject Invalid Domain

Actual HTTP execution of the Flask implementation

POST /threats

HTTP 400 | Expected 400 | 2026-10-20T20:32:17.943883+00:00

Request JSON


```
{
  "threat_type": "Domain",
  "threat_value": "https://invalid/path",
  "description": "Invalid",
  "severity": "High",
  "source": "Local test"
}
```


Response JSON


```
{
  "status": "error",
  "message": "Invalid threat indicator",
  "errors": {
    "threat_value": "'https://invalid/path' is not a valid domain name
(expected something like malicious-site.com, without http:// or a path)"
  }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Delete Log Record

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 200. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Delete Log Record

Actual HTTP execution of the Flask implementation

DELETE /logs/30

HTTP 200 | Expected 200 | 2026-10-20T20:32:17.954884+00:00

Request JSON

```
null
```

Response JSON


```
{
  "status": "success",
  "message": "Log record 30 deleted successfully",
  "data": {
    "log_id": 30,
    "user_id": 1,
    "action": "TEAM_EVIDENCE",
    "description": "Common report evidence run",
    "timestamp": "2026-10-20 20:32:11",
    "severity": "Information",
    "related_service": "Users"
  }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

Delete Threat Indicator

Omar Tabsh

Actual HTTP evidence viewer. Expected and observed HTTP 200. Native Postman evidence for Omar is still to be added to the final team version.



Omar Tabsh Delete Threat Indicator

Actual HTTP execution of the Flask implementation

DELETE /threats/1

HTTP 200 | Expected 200 | 2026-10-20T20:32:17.966884+00:00

Request JSON

```
null
```

Response JSON


```
{
  "status": "success",
  "message": "Threat indicator 1 deleted successfully",
  "data": {
    "threat_id": 1,
    "threat_type": "Domain",
    "threat_value": "sprint-evidence.example",
    "description": "Confirmed development indicator",
    "severity": "Critical",
    "source": "Team local test",
    "date_added": "2026-10-20 20:32:11",
    "last_updated": "2026-10-20 20:32:11"
  }
}
```


Development tokens used. This is an HTTP evidence viewer, not a Postman screenshot.

GitHub API History Capture 1

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

Actual authenticated GitHub API data displayed in the local viewer. Consecutive capture section.

Team GitHub Commit History

Authenticated GitHub REST API response for branch jad-moharam

https://api.github.com/repos/mohannad-gh/MohannadAlGhourayeb-JadMoharam-OmarTabsh/commits?sha=jad-moharam

Commit	Author	Commit message
f0f9959	Jad Moharam	Implement Jad Moharam services 3, 8 and 9 with all 18 APIs
f2824d8	Jad Moharam	Integrate Omar Tabsh services and shared team interfaces
db97c2f	Omar Tabsh	Document Omar Tabsh's APIs and the team integration contract Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
a4b329a	Omar Tabsh	Add run/dev-token scripts, database schema and Postman collection Postman collection covers all 18 APIs of services 4-6 with positive and negative cases (validation, duplicates, 401/403/404/405). Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
	Omar	Implement Security Reports service (Service 6) Six report endpoints computed from live data fetched through the other services' APIs using

GitHub API History Capture 2

Mohannad Al Ghourayeb | Jad Moharam | Omar Tabsh

Actual authenticated GitHub API data displayed in the local viewer. Consecutive capture section.

250293d	Omar Tabsh	the caller's token. Includes end-to-end tests running the threat, logging and reports services together over HTTP. Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
07f06bf	Omar Tabsh	Implement Security Audit Logging service (Service 5) POST /logs for administrators and trusted services, retrieval, search by user/date/action/severity/service, logs by user and admin-only deletion. Logs are immutable: no update route and a database trigger blocks UPDATE. Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
2a06e9f	Omar Tabsh	Implement Threat Intelligence service (Service 4) POST/GET/PUT/DELETE /threats, GET /threats/{id} and GET /threats/search with IP, domain and file hash validation, duplicate indicator rejection (409), role-based access and automatic audit logging of changes and violations. Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
bcc713a	Omar Tabsh	Add shared config, JWT authentication, response format and audit client Shared building blocks for all services: role constants, JWT verification with require_roles(), the agreed audit event vocabulary, the audit client that sends events to POST /logs with the inter-service key, and JSON error handling. Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
a620bce	Jad Moharam	Set up Flask and SQLite project with service requirements checklist
67428ca	mohannad-gh	Initial commit

This is a view of actual GitHub API data. It is not the GitHub website interface.

Current Real Login Integration Verification

Jad Moharam

The updated branch merges the latest main and Mohannad Users and Assets while preserving their authorship. A shared launcher creates the real SQLAlchemy tables before starting the nine available services. Jad supports their id primary keys and the earlier fixture key names. His endpoint business logic is retained.

The updated full suite passed 220 tests. The new integration test sends 33 actual HTTP requests using bcrypt password login for all four roles, then exercises Incidents, Notifications, Playbooks and Omar reports. It also checks invalid passwords, suspended accounts, stale roles and database foreign key consistency.

Actual sanitized execution evidence is included in evidence/real-team-integration.json. Passwords and bearer tokens are redacted. The test roles are provisioned locally after registration; no public administrator promotion endpoint was introduced. The result is HTTP integration evidence, not a native Postman capture.

Remaining teammate work includes vulnerability lifecycle/permission review, audit production, notification producers, SQLAlchemy foreign key enforcement and native Postman evidence. The actual vulnerability report now returns 200. The branch is prepared for teammate review; main is not merged automatically.

Real Integration Actual HTTP Results 1

Jad Moharam

Request	Actual status	Response message
1 POST /users/register	201	User registered successfully
2 POST /users/login	200	Login successful
3 POST /users/register	201	User registered successfully
4 POST /users/login	200	Login successful
5 POST /users/register	201	User registered successfully
6 POST /users/login	200	Login successful
7 POST /users/register	201	User registered successfully
8 POST /users/login	200	Login successful
9 POST /users/login	401	Invalid username or password
10 POST /assets	201	Asset registered successfully
11 GET /assets/search?criticality=High	200	Asset search completed successfully
12 POST /vulnerabilities	201	Vulnerability registered successfully
13 POST /incidents	201	Incident created
14 PUT /incidents/1/assign	200	Incident assigned
15 GET /notifications/user/2	200	Notifications retrieved
16 GET /incidents/1	200	
17 GET /incidents	403	User does not have permission to perform this operation

Real Integration Actual HTTP Results 2

Jad Moharam

Request	Actual status	Response message
18 PUT /incidents/1/status	200	Incident status updated
19 PUT /incidents/1/status	200	Incident status updated
20 PUT /incidents/1/status	200	Incident status updated
21 PUT /incidents/1/status	200	Incident status updated
22 POST /incidents/1/notes	201	Investigation note added
23 POST /playbooks	201	Playbook created
24 POST /playbooks/1/actions	201	Response action added
25 PUT /actions/1/status	200	Response action updated
26 PUT /actions/1/status	200	Response action updated
27 GET /reports/summary	200	Security Summary Report generated successfully
28 GET /reports/assets	200	Asset Report generated successfully
29 GET /reports/incidents	200	Incident Report generated successfully
30 GET /reports/vulnerabilities	200	Vulnerability Report generated successfully
31 POST /users/login	403	Account is inactive or suspended
32 GET /incidents/1	401	Authenticated account is missing or inactive
33 GET /playbooks/1	401	Account role changed; log in again

Users Assets and Vulnerabilities Database Entities

Mohannad Al Ghourayeb

These are the actual SQLAlchemy model fields supplied by Mohannad, initialized in the same SQLite file by the team launcher. The full executed DDL is in sql/schema_team_actual.sql. Relationships from assets.owner_id and vulnerabilities.asset_id are currently logical integer links rather than declared SQLAlchemy foreign keys; teammate enforcement review remains necessary.

Entity	Actual fields	Relationships and review
users	id PK; full_name, username unique, email unique, password_hash, department, role, account_status, created_date, last_login_date	Referenced by Jad reporter, analyst, recipient, author and creator IDs
assets	id PK; asset_name, asset_type, ip_address, department, owner_id, criticality, status, created_date	owner_id logically references users.id; Jad incident asset_id has a database FK
vulnerabilities	id PK; title, description, cve_id, asset_id, severity, cvss_score, status, remediation, discovered_date, updated_date	asset_id logically references assets.id; required lifecycle, permissions and field coverage need teammate review